

LodusPop: An Environment-Driven Multi-Scale Simulation Framework for Urban Population Dynamics, Service Resilience, and Flood-Aware Mobility

Gabriel Fonseca Silva, André da Silva Antonitsch and
Soraia Raupp Musse

*Virtual Humans Laboratory (VHLab) - School of Tecnology, Pontifical Catholic University of Rio Grande do Sul (PUCRS), Av. Ipiranga, 6681 - Partenon, Porto Alegre, 90619-900, RS, Brazil.

*Corresponding author(s). E-mail(s): fonseca.gabriel@pucrs.br;
Contributing authors: andre.antonitsch@edu.pucrs.br;
soraia.musse@pucrs.br;

Abstract

Urban population simulations can support the comparison of planning and emergency-response alternatives, but many approaches require detailed individual trajectories or remain tied to one application domain. This thesis presents LodusPop, an environment-driven, multi-scale simulation framework in which locations request population flows and populations are represented as heterogeneous blobs that can split, merge, move, and change state. The framework is extended with dynamic node availability and dependency constraints while retaining domain behavior in plugins. Its applicability is examined through published mobility, large-event, and epidemic studies and through new experiment families concerning inpatient care, dialysis-service disruption, routine activity visits, flooding, destination adaptation, and flood-aware commuting in Porto Alegre, Brazil. The experiments use both neighborhood-level and census-sector home representations, repeated and matched random seeds where appropriate, and domain-specific conservation and integrity checks. Results show how service coverage depends on capacity, spatial distribution, recovery time, and infrastructure dependencies, while flood experiments distinguish activity visits that can be rerouted from work and school trips with less substitutable destinations. Complementary immersive and origin–destination visualizations illustrate ways

to inspect simulation outputs, but are not treated as behavioral validation. The results are comparative outcomes under explicit assumptions rather than forecasts. Together, the studies demonstrate that LodusPop supports a broad family of auditable urban population scenarios without promoting application-specific capacities, queues, or adaptation rules into the core population model.

Keywords: Environment-driven simulation, Multi-scale modeling, Human mobility, Health-service resilience, Flood-aware mobility, Urban population dynamics

1 Introduction

1.1 Motivation and Problem

Recent crises have underscored the critical role of population modeling in urban planning and crisis management. Cities worldwide have faced urgent challenges in managing mobility, enforcing social distancing measures, and regulating the use of public spaces such as malls, supermarkets, and health services[1–3]. Similar challenges also emerge in other contexts, including disaster response, large-scale events, and climate-related risks[4–7]. These scenarios highlight the importance of simulation systems that can support the evaluation of urban policies and decisions, providing insights into how individuals and groups interact with the built environment[2, 5].

Simulation models are central to supporting evidence-based urban planning, from optimizing public transportation and allocating residential areas to improving the overall resilience of city systems [4, 5]. Human mobility is particularly relevant, forming the basis of research in epidemic control, traffic simulation, and socio-economic studies [6, 7]. Existing methods span microscopic approaches that track individual agents [8–11], macroscopic approaches that capture aggregated flows [12–14], and hybrid models that combine both perspectives [15–18]. In this thesis, our focus is on urban population modeling and mobility-oriented simulations across these scales, rather than on high-fidelity, collision-level crowd simulation. Despite the significant advances of existing methods, two major challenges remain for such population-dynamics platforms: i) integration across scales—connecting individual-level routines with population-wide patterns in a consistent framework [19, 20]; and ii) dependence on detailed data—most population-driven models rely on granular behavioral or mobility data, which is often unavailable or impractical to collect due to privacy and scalability concerns [19, 20].

1.2 Research Gap

Beyond routine mobility, an urban simulator intended for comparative planning must represent changes in the environment itself. A clinic may close, a home or point of interest may become exposed to floodwater, and the failure of a water-treatment facility may make dependent services unavailable. The population response also varies by domain: one trip may be suppressed, another may wait for capacity, and another may be redirected to a safe alternative. Existing mobility and service simulations

frequently embed such assumptions in a domain-specific model, limiting reuse and obscuring the boundary between the general simulation mechanism and the policy being tested.

This thesis therefore considers a third challenge in addition to integration across scales and dependence on detailed data: representing availability, capacity, disruption, and adaptation through explicit extensions while preserving a stable population model. Dynamic node availability and dependency constraints are treated as general properties of the environment. Health-service queues, flood exposure, destination substitution, and commute recovery remain plugin policies and, consequently, visible scenario assumptions.

To address these challenges, we introduce LodusPop, a flexible framework for simulating urban populations across multiple spatial and temporal scales. Instead of focusing on individual decision-making and detailed trajectory planning, LodusPop targets macro- and meso-scale population dynamics: environments actively request movements from aggregated population units. This environment-driven paradigm reduces reliance on fine-grained data while still capturing key patterns in aggregate flows and occupancy. Populations are represented through blobs – arbitrarily sized groups of individuals with defined attributes and routines – that can split, merge, and evolve during a simulation. This abstraction balances computational efficiency with the ability to represent heterogeneous population behaviors across scales ranging from neighborhoods to entire cities, while still providing hooks for micro-style routines when localized detail is needed.

1.3 Research Questions and Objectives

The general objective is to develop and evaluate a reusable framework for modeling heterogeneous urban populations under routine conditions, policy interventions, capacity limits, and environmental disruption. The thesis is organized around four research questions:

RQ1: How can an environment-driven, blob-based representation express heterogeneous population dynamics at neighborhood and census-sector spatial granularities?

RQ2: How can plugins, dynamic node availability, and dependency constraints represent domain-specific capacity, disruption, and adaptation while preserving a stable core model?

RQ3: What does the application of this common framework to mobility, public-health, health-service, and flood scenarios demonstrate about its versatility and practical limits?

RQ4: Which provenance, replication, validation, and visualization practices are necessary to interpret the generated outcomes as auditable scenario comparisons rather than forecasts?

To answer these questions, this thesis defines the LodusPop environment, population, and routine abstractions; retains the published experiments on random movement, large-scale events, epidemic contagion, vaccination, and computational

performance; and extends the evaluation with inpatient-care, dialysis-service, routine-activity, flood, destination-adaptation, and flood-aware commute experiments. Complementary visualization studies examine how aggregate outputs can be inspected without being confused with behavioral validation.

1.4 Thesis Contributions

The main contributions of this thesis are as follows:

- We propose an environment-driven modeling paradigm that shifts decision-making from individuals to the environment, reducing data requirements while remaining suitable for multi-scale urban mobility and policy analysis.
- We introduce the blob abstraction, enabling flexible and scalable representation of populations that adapt to data availability and simulation needs.
- We design LodusPop, a multi-scale simulation framework that integrates GIS-based environments with a blob-based population model within a unified architecture.
- We extend the environment with cause-aware node availability and dependency constraints, allowing direct disruption, recovery, and cascading service unavailability to be represented consistently.
- We evaluate LodusPop through cumulative case studies in Porto Alegre covering routine mobility, large-scale events, epidemic control, inpatient and dialysis services, flooding, destination adaptation, and commuting.
- We synthesize immersive and origin–destination visualization approaches, clarifying their role in exploring rather than validating simulation outcomes.

Together, these contributions establish LodusPop as a practical and extensible framework for studying urban population dynamics. The generated results are comparisons between explicitly modeled scenarios; they are not forecasts of individual behavior, clinical demand, or future flood impacts.

1.5 Publications and Thesis Organization

The model and original experiment family are based on the *main_submission* manuscript. The inpatient-care lineage and hospital-capacity data originate in the LODUS-Health master’s dissertation by Souza [21]; the recent batches reported here are separate experiments and are not a republication of that dissertation’s results. The immersive visualization is collaborative published work [22], whereas the geographic flow tool is student-led work developed by Silva under the supervision of the thesis author [23]. These sources are integrated with explicit attribution and with a distinction between prior artifacts, recent simulation work, and the synthesis contributed by this thesis.

Section 2 reviews crowd, mobility, resilience, health-service, flood, and visualization research. Section 3 defines LodusPop. Section 4 preserves the original published evaluation. Sections 5, 6, and 7 introduce the shared recent methodology and the two new experiment families. Section 8 discusses output exploration, Section 9 synthesizes the evidence and limitations, and Section 10 concludes the thesis. Detailed input and simulator examples remain in Appendix A.

2 Related Work

Research on modeling population dynamics spans two central but historically distinct areas: crowd simulation and population mobility modeling. Crowd simulation emphasizes fine-grained representations of movement and interactions among individuals or groups, whereas population mobility focuses on larger-scale flows and statistical regularities in urban dynamics. Each domain has produced significant advances, but both exhibit limitations that motivated the development of LodusPop.

2.1 Crowd Simulation

Crowd simulation has long served as a foundation for understanding human movement in both indoor and outdoor environments, with models developed at microscopic, macroscopic, and hybrid scales.

Microscopic models treat every agent as an autonomous entity with decision-making capabilities, typically influenced by local environmental conditions. Classical examples include the Social Force Model [8], where pedestrian motion emerges from forces balancing personal goals with environmental and social interactions, and ORCA [24], which guarantees collision-free trajectories through reciprocal velocity obstacles. BioCrowds [25] introduced a space-discretization strategy where agents compete for territory, naturally generating emergent Voronoi-like partitions. Later studies aimed to add heterogeneity by embedding psychological or emotional states into agent behavior. For example, Pelechano et al. [26] and Durupinar et al. [27] integrated factors such as stress, emotion, and cultural variation, while Neto et al. [28] modeled emotional contagion based on personality traits. These approaches achieve remarkable behavioral variety, but they depend heavily on detailed behavioral datasets and quickly become computationally expensive when scaled up to large urban populations.

In contrast, macroscopic models treat crowds as collective entities, often described through density fields or flow equations. Hughes [12] and Treuille et al. [13] pioneered continuum models where population density evolves through potential fields defined over the environment, while proposed Antonitsch et al. [17] BioClouds, aggregating agents into homogeneous “clouds” competing for space based on density and velocity preferences. Such formulations shift part of the steering to environment-side representations and significantly improve computational scalability, but they sacrifice the diversity of individual behaviors.

Hybrid models attempt to combine these two perspectives. For example, Legion [29] dynamically adapts the level of detail based on environmental complexity, allowing legions of agents to split or merge as situations demand. Xiong et al. [15] proposed spatially partitioned hybrid simulations that combine ORCA [24] in high-interest regions with continuum models elsewhere, while Lemonari et al. [30] introduced MPACT, a mesoscopic framework that uses learned profiles to capture intermediate-scale behaviors. These hybrids achieve flexibility, however, the primary focus of control remains on the population side. Agents query these fields and detailed data is often required for accurate calibration.

Recent advances explore ways to alleviate this dependency through data-driven modeling, where behavior is learned from observed or simulated trajectories rather than being entirely hand-crafted. For instance, machine learning has been used to embed agent dynamics into low-dimensional manifolds, enabling multiscale reconstruction of crowd flows [31]. Deep reinforcement learning approaches, such as those proposed by Li et al. [32], combine learned navigation strategies with anisotropic environmental fields to reduce computational costs in large spaces. In many of these studies, policies are learned from large numbers of simulated episodes rather than from real mobility datasets. Behavior is governed by fitted models whose data requirements center on simulated experience rather than fine-grained observational traces, and the underlying agent-based perspective remains unchanged.

In summary, crowd simulation excels in behavioral realism and the reproduction of fine-scale interactions. However, the reliance on detailed micro-data and high computational costs make it difficult to deploy at the scale of entire cities or to apply when granular behavioral data is unavailable.

2.2 Population Mobility

Population mobility studies emerged from statistical and large-scale analyses of human movement patterns. Unlike crowd simulation, which centers on the agent, mobility modeling typically abstracts away individuality to emphasize flows across geographic space and time.

A consistent finding in this field is that human displacements follow heavy-tailed distributions. Brockmann et al. [33] showed that the circulation of banknotes in the United States resembled Lévy flights, while González et al. [34], analyzing cell phone data, demonstrated that most individuals repeatedly visit a small set of locations. Song et al. [35] formalized this as a combination of exploration (discovering new places) and preferential return (frequent revisits), and Alessandretti et al. [20] refined these ideas with hierarchical “containers”, arguing that people conceptualize space in nested levels such as neighborhoods, cities, and regions. These statistical regularities provide strong predictive power, but they are generally limited to describing long-term patterns rather than simulating short-term interventions or local policies.

The COVID-19 pandemic highlighted the importance of connecting mobility with public health. Studies such as Jamshidi et al. [36] examined the effect of density and mobility restrictions on infection rates, while Hernández-Orallo et al. [37] used urban simulations to identify risk hotspots like train stations and plazas. Kraemer et al. [38] quantified how mobility and control measures influenced epidemic spread in China. These works demonstrated the societal relevance of mobility modeling but remained largely domain-specific, oriented toward epidemiology rather than more general urban planning applications.

Frameworks for simulating mobility have also emerged. Silva et al. [39] introduced a framework that integrates environmental data with macroscopic BioClouds to simulate contagion scenarios. Although innovative, its reliance on homogeneous clouds and a narrow focus on epidemics limited its applicability. More recent frameworks increasingly incorporate machine learning and multi-scale techniques [6,7,16], but they

remain strongly tied to data availability and typically adhere to the population-driven logic of traditional mobility models.

Recent contributions push the field toward more interpretable, adaptable, and AI-enhanced approaches. Liu et al. [40] proposed GraviGBM, which blends gravity-based migration models with gradient-boosted machine learning, balancing predictive accuracy and interpretability. Other work has used symbolic regression to uncover interpretable mobility laws directly from data, identifying power-law patterns in flows [41]. On the simulation side, new agent-based frameworks integrate large language models with mobility environments, as in CAMS [42], enabling autonomous agents to generate plausible movement patterns without pre-defined geospatial inputs.

Additionally, several general-purpose urban simulation frameworks are widely used. MATSim [43] is a large-scale agent-based framework for transportation planning that simulates individual travel demand and traffic assignment through iterative optimization. SUMO [44] is a microscopic traffic simulator designed for modeling vehicles and road networks, offering fine-grained mobility but limited population abstraction. GAMA [45] provides a flexible multi-agent platform where users can model a wide range of urban and environmental processes with rich GIS integration.

In summary, mobility research provides robust statistical frameworks and promising AI-driven methods, but most approaches are either too aggregated for dynamic, multi-scale interventions or remain dependent on detailed mobility data.

2.3 Positioning LodusPop Between Crowd Simulation and Mobility Models

The works reviewed above point to a gap between crowd simulation and population mobility models. On one side, crowd simulators are capable of reproducing rich local interactions and complex behaviors, but they become expensive at city scale and often presuppose detailed, behavior-level data for calibration. Mobility models, in turn, describe flows at coarser levels and scale more easily to large regions, but offer limited support for representing how specific locations, policies, or events structure movement over time in a way that remains explicit and manipulable.

LodusPop is designed to bridge this gap by making the environment an explicit driver of population dynamics while retaining heterogeneity in the population. Routines are attached to regions and points of interest instead of individuals, so locations express which types of population they demand or release at different times. For example, schools requesting children from nearby residential areas, hospitals attracting patients with particular attributes, or shelters drawing specific profiles subject to capacity constraints. Populations are represented as blobs of variable size that carry typed, traceable attributes and can split when only a subpopulation participates in an action or undergoes a change of state, and merge again when compatible groups share the same location. In this way, a single simulation can move smoothly between aggregate flows and more detailed descriptions where fine tracking is important, without losing consistency of attributes over time.

The dynamics of LodusPop are expressed in a compact yet extensible action formalism that covers both movements in the environment and changes in traceable characteristics. Routines attached to locations are written as sequences of such actions,

which the simulator interprets to update blobs and their distribution across the environment. Additional processes such as disease transmission, recovery, vaccination, or participation in specific events can be introduced as new action types together with their consumption rules, implemented as plugins that integrate with the same core abstraction. As a result, LodusPop can combine different mobility and process models within a unified environment-driven representation, reusing the same engine to explore diverse scenarios while maintaining a clear separation between the core simulator and domain-specific mechanisms.

2.4 Capacity and Accessibility in Urban Health Services

Health-service planning combines temporal demand, geographically distributed facilities, and resources that cannot always be substituted. Prior studies address hospital admission prediction [46], scheduling of non-critical admissions [47], bed-allocation optimization [48], and integrated simulation–optimization of emergency-care pathways [49]. These approaches demonstrate the value of explicitly representing capacities, lengths of stay, queues, and routing policies. They also show why an unconstrained flow is not equivalent to an operational admission policy: accepting all demand can reveal a latent deficit, whereas enforcing capacity creates waiting populations and delayed service.

LODUS-Health applies urban simulation to hospital-bed occupancy in Porto Alegre and provides the dataset and conceptual lineage for the inpatient experiments in this thesis [21]. Its contribution is treated as prior work rather than as a result of the present experimental batches. The recent implementation separates observed facility capacities from configurable demand and routing assumptions, and compares an unconstrained overflow counterfactual with a capacity-enforcing queue. This separation is important because it exposes which conclusions concern available beds and which depend on a synthetic allocation or demand profile.

Recurring services add further temporal structure. Dialysis patients require repeated treatments within a schedule, and service availability may depend on both clinic capacity and supporting infrastructure. Consequently, a general urban framework must permit domain plugins to express recurring demand and service policy without treating a hospital bed or dialysis chair as a universal property of every point of interest.

2.5 Infrastructure Disruption, Dependencies, and Urban Resilience

Urban resilience research increasingly combines human mobility with infrastructure and community context [6, 50]. For simulation, a disruption is more than a change in travel demand. It can remove an origin, destination, or supporting service; its effect may persist or recover; and one failure may make other nodes unavailable. A useful representation must therefore distinguish a node’s direct state from its effective state after dependencies are evaluated.

This thesis represents direct disruptions as named disable causes and represents dependencies through two rule forms: all listed prerequisites must be available, or at

least a specified number of nodes from a prerequisite set must be available. These abstractions cover, for example, a clinic requiring a particular support facility or at least one available facility from a group. They are intentionally narrower than a general infrastructure-flow model: they express service availability, not hydraulic, electrical, or transportation physics.

2.6 Flood-Aware Mobility and Destination Adaptation

Human-mobility data are widely used to characterize exposure, evacuation, access, and recovery in urban disruptions [6]. In a location-based simulator, however, a flood-aware trip requires explicit semantics for both ends of the movement. An unavailable home can suppress demand or force a temporary origin; an unavailable destination can cancel a trip or trigger substitution; and a return can fail after conditions change. These alternatives produce different outcomes even when the same nodes are exposed.

The experiments in Section 7 distinguish routine activity visits from work and school attendance. Activity demand can be redirected to a nearby enabled point of interest of the same type. Workplaces and schools are treated as assigned destinations in the reported commute scenarios, so their loss is not assumed to be solved by the same substitution rule. This distinction makes adaptation a testable scenario policy rather than an implicit claim that all urban destinations are interchangeable.

2.7 Simulation and Origin–Destination Flow Visualization

Simulation output is commonly summarized through time series, maps, and origin–destination matrices. Immersive visualization can place node occupancy and population displacement within a three-dimensional urban context. Teixeira et al. [22] implemented such a prototype for crowd-contamination outputs, including selection of regions of interest, temporal inspection, and capacity-oriented summaries. Its demonstration used randomly generated events; it established technical feasibility, not the behavioral or predictive validity of the underlying simulation.

The geographic flow tool developed by Silva under the thesis author’s supervision takes a complementary approach [23]. It supports interactive incoming and outgoing flows, geographic aggregation, demographic filters, coordinated charts, and in-browser analytical queries. The principal study uses 2021 census commuting data from England and Wales, while a simulated Porto Alegre dataset demonstrates configuration portability. The centroid-to-centroid lines describe aggregate relations and do not reconstruct traveled paths.

These works motivate a common output perspective for LodusPop: temporal node states describe availability and occupancy, movement logs describe individual blob transfers, and aggregated origin–destination tables describe flows between places or territorial units. Visualization makes these records easier to inspect, but agreement within a visualization is an internal-consistency property rather than external validation.

2.8 Research-Gap Synthesis

The literature provides strong models for detailed crowds, statistical mobility, health-service allocation, infrastructure resilience, and geospatial visualization. The unresolved integration problem is to reuse a common heterogeneous population representation while allowing locations, capacities, failures, dependencies, and adaptations to vary by research question. LodusPop addresses this gap through three boundaries. First, blobs and population templates belong to the core model. Second, effective node availability belongs to the environment. Third, service demand, capacity, queues, flood exposure, fallback selection, and return behavior belong to plugins. The experiments that follow evaluate whether this separation is sufficiently expressive across several domains; they do not claim that one plugin policy is a universal behavioral model.

3 Proposed Model

Population mobility area studies the large-scale flow of a population in a given geographic space, describing this population as a whole instead of its individuals [33, 34]. This approach may have different degrees of granularity, as populations can be studied in a given district, city, state, country, continent, or even the entire world and in different time scales, as data can be hourly, daily, monthly, yearly, and so on. In this work, we propose LodusPop, a flexible framework for multi-scale population simulation. Instead of representing individuals or fixed groups, LodusPop models populations as variable-sized units, each with defined characteristics and behavioral patterns. This design enables seamless adjustment of simulation detail, adapting to both the availability of data and the requirements of the study.

The simulation process relies on data to function effectively. LodusPop can utilize real-world datasets while offering the flexibility to customize population parameters. Indeed, LodusPop was designed to simulate population mobility allowing the integration of data from diverse sources, such as real-world datasets and programmed scripts. Figure 1 illustrates the potential data sources and their integration within the components of our proposed model. On the left, we can see possible sources of data, which are specified in the center, that can be used in the platform. On the right, the core components of LodusPop are depicted: the environment, defining the simulated space; the population, representing entities moving within the environment; and the simulation behavior, responsible for generating actions that influence the population. These components are detailed in the subsequent sections. As an example, a simulation designer can define a population and model their behavior using a combination of real-world and synthetic datasets. Environmental data can be sourced from Geographic Information Systems (GIS) databases like OpenStreetMap. Population profiles may be derived from census data (Real Population) or generated synthetically (Synthetic Population) to fit the specific needs of an experiment. Additionally, data describing population behavior, such as routines, can be obtained from sources like Origin-Destination Matrices, GPS tracking services, or user-generated inputs.

Population mobility frameworks typically operate across multiple spatial scales, such as neighborhoods, cities, and states. In this work, we seek to integrate strategies from crowd simulation into human mobility modeling, particularly the ability of

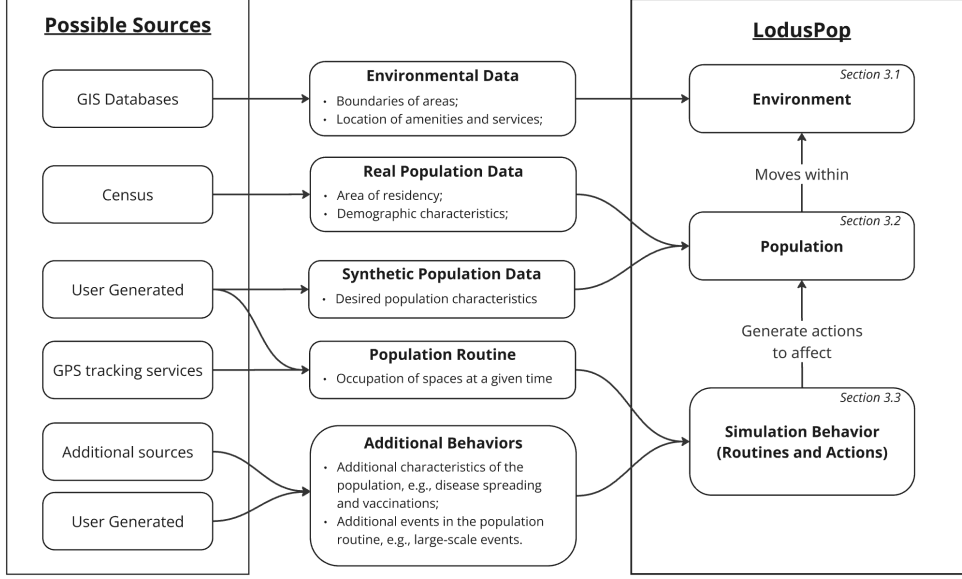


Fig. 1 Flowchart illustrating the integration of data sources into the LodusPop framework. Inputs such as GIS databases, census data, user-generated data, and GPS tracking services are processed into geographic data, population characteristics, and behavioral routines. In our proposed model, routines consist of actions that influence population behavior, driving their movement through the environment.

macroscopic models to abstract individual behaviors [13, 14]. To achieve this, we propose representing populations as blobs—aggregated groups of individuals of varying sizes. In this context, Blob is an abstract structure, similar to a cloud or a legion [17], but designed to represent larger and more heterogeneous populations. Unlike BioClouds and Legion, which assume homogeneous population clusters, our blobs can encompass diverse individual and group attributes and evolve within environments containing locations that demand specific population profiles.

Furthermore, our environment design builds upon the approach proposed by Silva et al. [39]. However, our model introduces greater flexibility, allowing regions of the simulated environment to represent different semantic entities beyond the fixed hierarchy of countries, states, regions, and cities, enabling more adaptable and context-specific representations.

In LodusPop, we can define a set of characteristics for each blob, representing a population of a specific profile distributed in the environment and changing temporally and spatially in the simulation. For instance, one group, representing a blob, might consist of children going to school in the morning and returning home in the afternoon, illustrating spatial movement within a city. Another example involves changes in population parameters, such as healthy workers commuting to their jobs in the morning, with some individuals becoming sick during the day, resulting in healthy individuals returning home and sick individuals heading to a hospital, in this case representing a split in the blob structure.

Our proposed model considers the following elements to provide population simulation, which will be described in the next sections.

- a) An environment modeled as a graph of regions and points of interest that the population can occupy;
- b) The population is modeled as blobs that can split and merge and can contain characteristics of the population; and
- c) a set of population movement actions and routines to describe the desired population behavior during the simulation.

3.1 Environment

To adequate our population mobility simulation to different spatial scopes, we model the environment as an abstract graph composed of *regions* and *points of interest*. The regions (R) represent areas of physical space in the virtual world that can contain points of interest (P), which represent locations that motivate human travel and that can have population groups (i.e., blobs B). When setting up a simulation, users define the spatial scope that R and P represent. For example, in a city simulation, R could correspond to districts, while P might represent markets, malls, hospitals, residential buildings, and commercial centers.

A region R is represented as a set of points of interest $\{P_1, \dots, P_m\}$. Each point of interest P_i stores the blobs currently located there, denoted $\mathbf{B}_{P_i}$ and a local routine RoP_i that specifies which actions occur at that location over time as the simulation evolves (3.3). Table 1 summarizes the notation used for regions and points of interest. Additionally, Appendix A outlines the process of defining the input data necessary for designing a simulation. Appendix A.1 specifically addresses the data required to model the simulation environment.

In addition to its blobs and routine, every point of interest has a time-dependent *effective availability*. Let $C_i(t)$ be the set of direct causes disabling P_i at step t , such as an off-cycle closure or flood exposure, and let $q_i(t) = 1$ exactly when $C_i(t)$ is empty. A point of interest may also declare a set D_i^{all} of prerequisites that must all be enabled and one or more threshold groups (k, G) requiring at least k enabled prerequisites from G . Its effective state is

$$e_i(t) = q_i(t) \wedge \bigwedge_{P_j \in D_i^{all}} e_j(t) \wedge \bigwedge_{(k, G) \in D_i^{min}} \left[\sum_{P_j \in G} e_j(t) \geq k \right]. \quad (1)$$

When no dependency rule is defined, $e_i(t) = q_i(t)$. Dependencies may be transitive, so effective states are recomputed until no state changes. Removing a cause from C_i does not enable the node while another cause remains, and satisfying a direct state request does not override an unsatisfied dependency. This cause-aware formulation avoids, for example, reopening a flooded facility merely because its scheduled closure ended.

Availability records whether a location can participate in an action; it does not prescribe a universal population response. A plugin may suppress a request from a disabled origin, reject a disabled destination, retain demand in a queue, select a

fallback, or reroute it. These alternatives are domain policies and are kept outside the core environment definition.

Indeed, this characterizes an inversion of responsibility compared to what usually happens in population simulations. In many agent-based approaches, populations decide where to go based on individual preferences or probabilistic rules, requiring granular data about individual movements and locations. Even in field-based models, where motion is guided by environmental potentials or vector fields, agents typically remain the entities that sample and follow these fields. In our case, the environment is responsible for decision-making regarding the occupation of the population during the simulation, i.e., locations ask for people instead of people choosing where to go. We believe that shifting the focus to the environment as the decision-maker minimizes dependence on granular individual-level data. This shifts part of the decision logic from individuals to points of interest and regions, and is designed to minimize dependence on granular individual-level data by leveraging aggregated or generalized information about population distributions and environmental conditions. For example, a P_m representing a market can request consumers, i.e., P_m has a local routine Ro_{P_m} , which defines that a certain population of determined characteristics will go to the market during the simulation. B_{P_m} represents all the population blobs that are currently on the market.

Other examples in the same region R can be a school that requests students, and hospitals that request patients. In these cases, school is a P_j in R , in which Ro_{P_j} is the local routine to go to the school and B_{P_j} is a specific population of students. Similarly, the hospital is also a P_k in R , and Ro_{P_k} is a local routine that should be executed at the hospital. Additionally, B_{P_k} is a particular population that goes to the hospital. Figure 2 presents a diagram of a LodusPop environment, representing regions, points of interest, and blobs. As humans generally tend to present cyclical behavior in their movement [34], repeating similar routines daily or weekly, a LodusPop simulation is also cyclical, where local routines Ro repeat every time cycle of simulation. The user defines the temporal scope of a cycle when designing the simulation. A cycle may represent an hour, a day, a month, etc.

3.2 Population

The population in LodusPop is represented by an abstraction of people into blobs B , each always located at some point of interest. These blobs describe the population in groups rather than tracking individuals. Formally, we denote a blob as $B = (TP, Tc, Sc)$ where TP is the total number of individuals in the blob, Tc is the collection of traceable characteristics, and Sc is the collection of sampled characteristics. Table 2 summarizes this notation and the associated terminology.

We distinguish two types of characteristics. *Sampled Characteristics*, collected in Sc , are attributes represented by their distribution within the blob. For example, a sampled characteristic *Age* may have categories *Child*, *Adult* and *Elder*, each associated with a certain number of individuals. By combining multiple sampled characteristics, we can jointly describe attributes such as age, level of schooling and employment status. *Traceable Characteristics*, collected in Tc , are attributes whose values are explicitly stored as a single value for the entire blob, rather than as a distribution

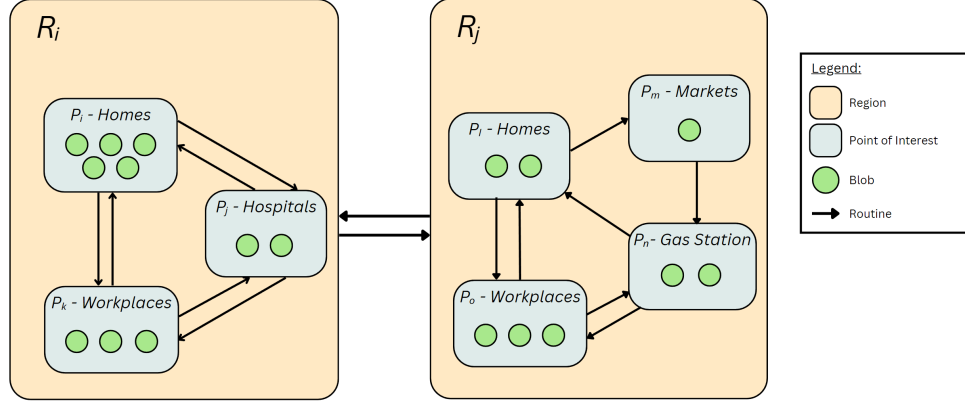


Fig. 2 A diagram of a LodusPop environment. The figure presents two regions, R_i and R_j , containing different points of interest P . Each point of interest may contain groups of the population (i.e., blobs), and routines are responsible for moving them in the environment during the simulation.

Table 1 Mathematical notation and descriptions of the elements defined in Section 3.1.

Element	Notation	Description
Region	R	Represent areas of physical space in the virtual world that can contain points of interest. A region R_i is defined as a set of P .
Point of Interest	P	Represent locations that the population can occupy as they move during a simulation. A P_i is defined by a collection of population blobs (B_{P_i}), its local routine (Ro_{P_i}), and its effective availability.
Direct disable causes	$C_i(t)$	Named, independently removable causes that directly prevent P_i from being available at step t .
Dependency constraints	D_i	All-of and minimum-of prerequisite rules used to derive effective availability.
Effective availability	$e_i(t)$	Boolean state obtained from the direct causes and the effective states of all prerequisites.

over categories. For instance, a traceable characteristic *Vaccination status* with values *Vaccinated* and *Unvaccinated* implies that each blob in the simulation is composed entirely of vaccinated or entirely of unvaccinated individuals.

These characteristics allow us to specify the population in more detail, e.g., elderly people who are currently vaccinated against a given disease or the number of adults who commute to work by bus. Following the previous examples, it becomes possible to track vaccination by age group in the simulation by identifying all blobs B with a *Vaccinated* status and their respective *Age* distributions. Figure 3 shows a diagram of a blob, including its characteristics and associated distributions. Appendix A.2 presents an example of the input data required to model the initial population in simulations.

As mentioned before, during the simulation, blobs in P_i are affected by the local routine Ro_{P_i} . Routines can select a specific population profile to be affected, e.g.,

moving only children to school, i.e., people from a particular sample attribute “Age” in the category “child” should go from P_i to P_j . This type of operation in the population B_i of a certain P uses a population template.

A population template T specifies which individuals are affected by an action and in what quantity. We represent it as $T = (T_Q, T_{Tc}, T_{Sc})$, which specifies i) the quantity of people to be affected, ii) an optional a set of predicates on traceable characteristics, and iii) is an optional set of predicates and distributions on sampled characteristics. When applied to a set of blobs, a template behaves as a filter that identifies the subset of individuals that satisfy all predicates and are counted towards T_Q .

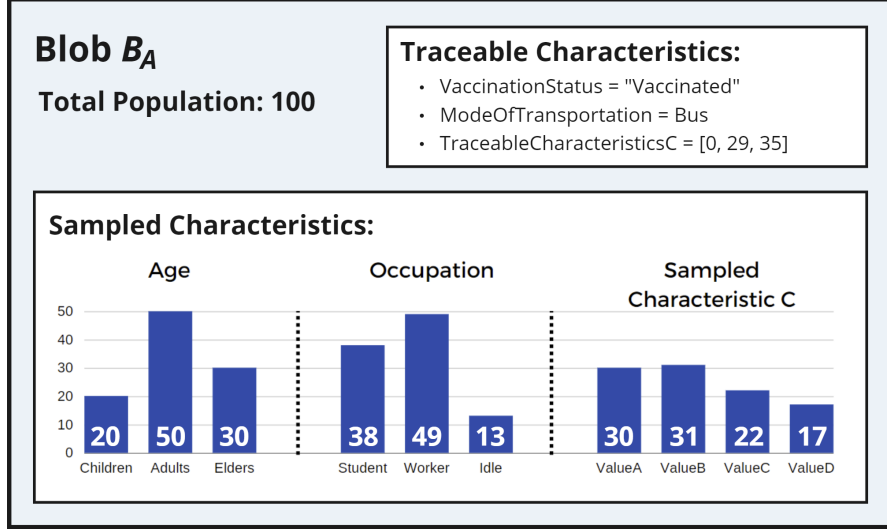


Fig. 3 Diagram of a Blob B_A containing 100 people, three traceable characteristics, and three sampled characteristics. Traceable characteristics describe the entire blob population and can have any value type. The distribution of each sampled characteristic sum should be equal to the total population of the blob.

Consider a blob B_j with two sampled characteristics: *i*) *Age*, which includes the possible values *Children*, *Adults*, and *Elders*, and *ii*) *Occupation*, with possible values *Student* and *Worker*. A template T that specifies *Age=Adults* in its collection of sampled characteristics will target only the specified group within B_j , in this case, only adults. If the template T does not include a filter for the second sampled characteristic (*Occupation*), all occupation categories in B_j are affected. The number of affected individuals in categories such as *Students* and *Workers* is determined proportionally based on their distribution within B_j . For instance, if B_j contains more students than workers, a larger proportion of students will be selected. This process continues until the total number of individuals defined in T is selected or until there are no remaining eligible individuals in B_j .

When a blob is operated using a population template T , it is possible that only a portion of its population may be affected, e.g., a blob B_i that has 75 people and only 20 are selected in T . If the operation is to move to another P , then such 20

people will create a new blob. This operation generates a split, i.e., the original blob will be separated into two smaller ones. In such an example, a template with quantity $T_Q = 20$ selects 20 people from B_i and splits it by creating a new blob B_j , which is moved to a different P . The same occurs when modifying a blob's Tcs . For example, another blob B_k has 90 *Healthy* people, and 20 are selected, using T , to be infected by a disease. The split occurs, and a *InfectionStatus* of B_l (newly created blob) is changed to *Infected*.

In addition to split operations, blobs can also be merged, combining their total populations TP . Each sampled characteristic of the resulting blob is the sum of the respective characteristics of the merged blobs. A merge can only occur between blobs currently in the same P and with the same values in all their traceable characteristics. This restraint allows us to merge only populations that share the same traceable features. Figure 4 presents a diagram of blobs affected by two templates, resulting in splits and merges. The first scenario, depicted at the top of the figure, demonstrates the relocation of a population. The second scenario illustrates changes in traceable characteristics of a population.

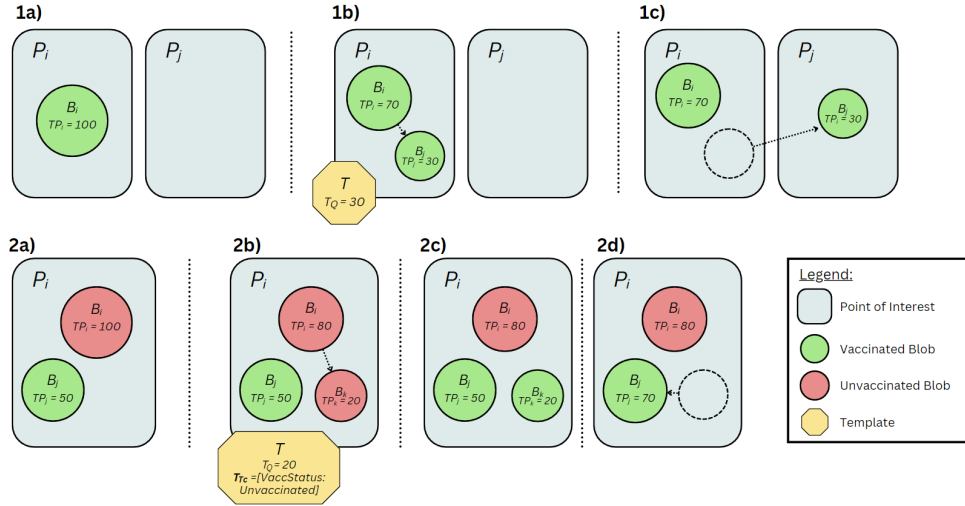


Fig. 4 Diagram presenting two scenarios of blobs being modified. The first scenario (1) (on the top) shows a population being moved to another location. In 1a, the initial blob B_i has a population of 100. In 1b, a template T selects 30 people, splitting B_i and creating B_j . In 1c, B_j is moved to P_j . The second scenario (2) shows a population's traceable characteristics changed. In 2a, there are two blobs, B_i containing 100 unvaccinated people (in red), and B_j containing 50 vaccinated people (in green). In 2b, a template T selects 20 unvaccinated people, affecting only B_i and splitting it into B_k . In 2c, the traceable characteristic *VaccStatus* of B_k is changed to *Vaccinated*. In 2d, B_k is merged into B_j since they now share the same characteristics and are in the same P_i .

Table 2 Mathematical notation and descriptions of the elements defined in Section 3.2.

Element	Notation	Description
Blob	B	Represent an abstraction of the population. A B_i is defined as its total population (TP_i), a collection of traceable characteristics (Tc_i), and a collection of sampled characteristics (Sc_i)
Traceable Characteristic	Tc	Characteristic of a blob defined as an identifier (i.e., a string) and its value (not restricted by type).
Sampled Characteristic	Sc	Characteristic with a categorical distribution over the blob population. Defined as a string identifier and a collection of its possible categories, each associated with a certain number of people.
Population Template	T	Specifies which individuals are affected by an action and in what quantity, operating as a filter to select a population from a B .

3.3 Routines and Actions

All behavioral dynamics in LodusPop are driven by routines, which associate discrete time steps with actions that operate on blobs. We consider a simulation organized into cycles of fixed length d steps. Within a cycle, time is indexed by an integer $t \in \{0, \dots, d - 1\}$, which we refer to as the cycle step.

A (local or global) routine is a finite set of pairs $R = \{(t, A)\}$, where t is a cycle step and A is an action to be executed at that step. For each point of interest P_i , the local routine is denoted Ro_i and specifies the actions that should be performed by P_i . The Global Routine GRo has the same structure but applies its actions to all points of interest in the environment.

Each action A is defined as $A = (N, T, V)$, where N is an identifier for the action type (e.g., MOV or CH); T is a population template as defined in Section 3.2, selecting the subset of a blob that will be affected; and V is a vector of additional values (parameters) used by the action. Table 3 summarizes the notation introduced in this section.

In the core implementation, LodusPop provides two built-in action types: *move population* (MOV), which moves a subset of a blob between points of interest; and *change traceable characteristic* (CH), which modifies the traceable characteristics of a subset of a blob.

A MOV action expects its parameter vector V to specify an origin and a destination point of interest. For example, the local routine of P_i may include the pair $(t = 15, A)$, where $A = (MOV, T_i, V_i)$, with $T_i = (T_Q = 20, T_{Tc} = \emptyset, T_{Sc} = \{Age = Child\})$, and V_i encoding P_i as origin and P_j as destination. At cycle step $t = 15$, this action selects 20 children from the blobs at P_i , splits them into a new blob, and moves that blob to P_j .

A CH action expects V to contain the name of a traceable characteristic and its new value. For instance, the local routine of P_j may include $(t = 25, A')$, with $A' = (CH, T_j, V_j)$, where $T_j = (T_Q = 20, T_{Tc} = \{Status = Unvaccinated\}, T_{Sc} = \emptyset)$, and V_j encodes the update “Status := Vaccinated”. At cycle step $t = 25$, this action selects 20 unvaccinated individuals from the blobs at P_j , splits them into a new blob,

and changes the traceable characteristic Status of that blob to Vaccinated. Figure 4 illustrates both the movement and characteristic-change scenarios.

The same action representation is used by the Global Routine *GRO*. For example, a MOV or CH entry in *GRO* is executed at the specified cycle step t for all points of interest simultaneously, enabling the modeling of global phenomena such as random walks or disease spread. Appendix A.3 provides an example of the input data used to encode both local routines and the Global Routine.

New action types can be added via plugins. A plugin registers an additional identifier N together with a function that consumes actions of that type. Plugin actions retain the same structure $A = (N, T, V)$. The simulation still uses the population template T to select the affected individuals and passes T and V to the plugin’s function. Appendix A.4 illustrates how plugins can be used to introduce custom action types into simulations.

Plugins may also schedule changes to node availability. A state-change action identifies one node, or a set of nodes by type and region, the state being requested, and the direct cause being added or removed. After the direct change, Equation 1 is reconciled transitively. Subsequent actions in the same simulation step observe the resulting effective state, consistent with the sequential action semantics described below.

The effect of this state on population behavior is deliberately defined by the consuming plugin. In the experiments of this thesis, plugins use effective availability to suppress demand at an unavailable origin, refuse admission to an unavailable service, select an enabled fallback facility, retain patients in a capacity queue, or reroute activity visits to a safe point of interest. Service capacities, queues, flood thresholds, patient stays, destination substitution, and commute lifecycles are therefore extensions of the action semantics rather than new fields of a blob or universal properties of every P .

To achieve repeating temporal behavior, the simulation iterates over cycle steps $t = 0, \dots, d-1$ within each cycle. At the beginning of each step t , the simulator collects all actions (t, A) from *GRO* and from every local routine. For each action, it:

1. uses the template T to select the affected population from the current blobs;
2. applies the action semantics (e.g., move or characteristic change), which may split existing blobs or create new ones; and
3. performs, at the end of the step, a merge operation at each point of interest, combining blobs that share identical traceable characteristics.

Figure 5 illustrates the simulation behavior over a single cycle. Appendix A.5 presents an example of a simulation script that manages the overall control of the simulation flow. In the current implementation, actions scheduled for the same cycle step are executed sequentially in the order in which they are listed in the input, and we apply the same ordering to actions in the global routine *GRO*. Each action is treated as an atomic operation: it selects a population according to its template based on the current blob configuration, applies the corresponding modifications, and immediately updates the state before the next action is evaluated. As a consequence, the same blob (or its descendants) may be affected by multiple actions within a single cycle step, but the later actions always see the state resulting from earlier ones in that step.

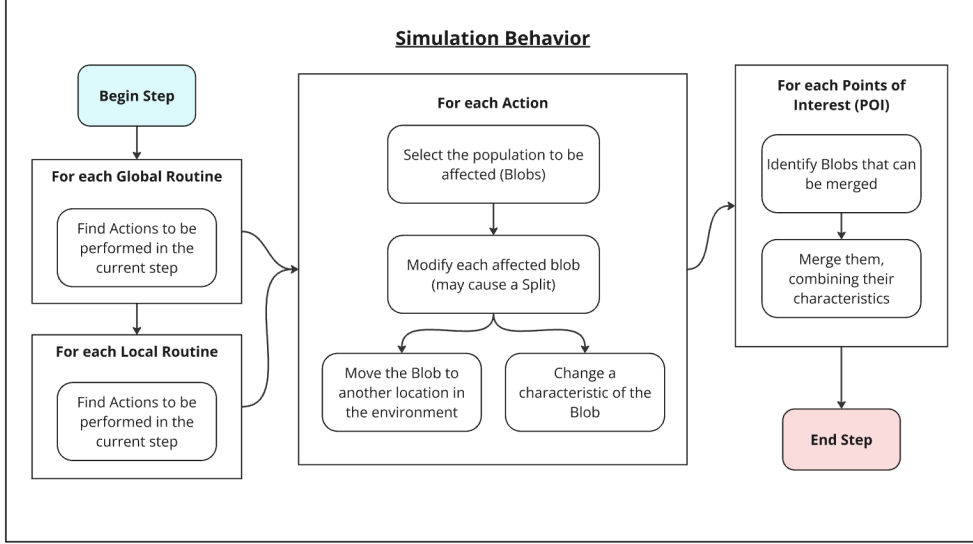


Fig. 5 Diagram of the simulation behavior, illustrating the identification and execution of actions from global and local routines. At each cycle step, all actions scheduled for that step are collected and executed sequentially in the order defined by the simulation input. Each action modifies blobs by moving them or changing their traceable characteristics and may cause splits; subsequent actions in the same step operate on this updated state. At the end of the step, blobs with identical traceable characteristics in the same point of interest are merged.

Table 3 Mathematical notation and descriptions of the elements defined in Section 3.3.

Element	Notation	Description
Local Routine	Ro	Model the repeating temporal behavior of a population B located in any given P . It is defined as $\{t, A_i\}^*$, where t is the time in the simulation that action A_i should be performed.
Global Routine	GRo	Modeling the global behavior of a simulation. Similar to a Local Routine, but it is applied to all Ps in the simulation
Action	A	Represents an action to be performed in the population, either moving or changing its characteristics. A A_i is defined as an identifier for the action type ((N_i)), a population template to select the population to be affected (T_i), and a collection of values to be used by the action (V_i).

4 Experimental Results

This section presents a series of experiments designed to evaluate the key aspects and capabilities of our proposed model, LodusPop. These experiments demonstrate the model’s versatility and effectiveness in simulating population dynamics under various scenarios, highlighting its ability to manage complex interactions between the environment and the population. Each experiment highlights specific features of the model:

1. **Random walk behavior** simulates daily routines based on the population’s profile (i.e., sampled characteristics) and the proximity of points of interest. For instance, students can be modeled to attend schools during the day, while adults visit nearby pharmacies or supermarkets (Section 4.2);
2. **Large-scale events** illustrate how disruptions to daily routines impact mobility and how multiple events can compete for the available population (Section 4.3);
3. **Infectious disease simulation and vaccination scenarios** model population behavior over extended periods, illustrating how traceable characteristics evolve and how the spread of a disease impacts the population. These scenarios also enable the analysis of dynamic population attributes, demonstrating how public health policies, such as vaccinations or lockdowns, influence disease transmission and population behavior (Section 4.4).

All experiments extend the basic functionalities of LodusPop by incorporating plugins that introduce additional action types and traceable characteristics, enabling precise tracking of population evolution throughout the simulations. Additionally, these experiments demonstrate the model’s capability to represent populations as abstract blobs of varying sizes, from coarse city-wide aggregates down to blobs that represent very small groups or even single individuals. Macro-scale dynamics are evident when large groups participate in large-scale events across the environment, resulting in changes to mobility patterns (Section 4.3), while finer-grained effects appear when a small subset of the population at a specific location is affected, such as during a disease outbreak (Section 4.4). The blob sizes are dynamically adjusted during simulations, ranging from entire neighborhood populations to single individuals. Together, these case studies validate the model’s performance and highlight its potential for practical applications in urban planning and public health.

4.1 Case-Scenario: Porto Alegre, Brazil

LodusPop, designed to work with real-world data, was tested to model the population behavior of Porto Alegre, Brazil. Following our proposed definitions for the environment (Section 3.1), we modeled each neighborhood of the city as a region (R) that contains a set of points of interest (P). Environmental data representing the boundaries of each neighborhood was obtained through a dataset made available by Porto Alegre’s city hall¹. Figure 6 presents the neighborhoods of Porto Alegre. All 94 neighborhoods of the city are included in our experiments. Four specific neighborhoods are highlighted for their additional routines, which are utilized in our large-scale experiments (Section 4.3).

We utilized the OpenStreetMap database² to extract the geographic coordinates (latitude and longitude) of buildings and amenities within Porto Alegre that could serve as points of interest for the population, such as schools and pharmacies. These points of interest were distributed across the city’s neighborhoods. Each region in the model includes the following Ps : *home*, *work*, *school*, *marketplace*, *pharmacy*, and *gas_station*. Additionally, some regions may feature other Ps , such as *shopping_malls*,

¹Neighborhood boundaries Shapefile are available at (under “Bairros (LC 12.112/16)”: <https://prefeitura.poa.br/carta-de-servicos/mapas-digitais-da-smamus>

²OpenStreetMaps is available at: <https://www.openstreetmap.org/>

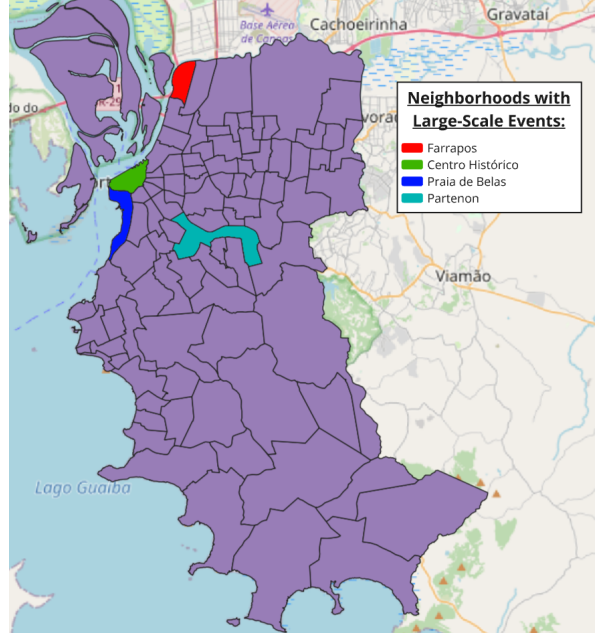


Fig. 6 The 94 neighborhoods of Porto Alegre, Brazil. All 94 neighborhoods of the city are included in all our experiments, with four specific neighborhoods highlighted for their additional routines, utilized in our large-scale experiments (Section 4.3).

hospitals, universities or stadiums. The modeled environment comprises 94 *Rs* and a total of 504 *Ps*. For the experiments described in this paper, each region contains at most one *P* of each type, such as one *home* and one *pharmacy* per neighborhood. This setup was dictated by the available data, which provided information on population distribution at the neighborhood level but did not specify finer-grained details within neighborhoods. However, this is not a limitation of the proposed model. When more granular data is available, simulation designers can enhance the environment by adding multiple home *Ps* within a single region and allocating neighborhood populations accordingly. Appendix A.1 provides an example of the syntax needed to inform data for modeling an environment in our simulations.

To model the population in our simulations, we collect data from governmental sources, including the most recent completed census at the time of this publication³. Census data provided citywide information on the total population, the number of economically active individuals (i.e., workers), the number of students, and the age distribution of the population. However, this information lacked specificity at the neighborhood level. To address this, additional data was gathered from city administration surveys, which included neighborhood-level details on total population and

³Census Conducted by the Brazilian Institute of Geography and Statistics (IBGE). The data is available at: <https://cidades.ibge.gov.br/brasil/rs/porto-alegre/panorama>

age distribution ⁴⁵. This information was used to define sampled characteristics for the population, with the following identifiers and categories:

- *Age*: Child, Young, Adult, and Elder;
- *Occupation*: Student, Worker, and Other.

Porto Alegre’s total population is 1,409,351. Since *occupation* data was only available at the city level, all neighborhoods were assigned the same distribution: *students* comprise approximately 21.575% of the population, *workers* 54.706%, and *other* 23.718%. The *age* distribution was mapped using the age pyramid, with each category representing a specific range: *child* (0–14 years), *young* (15–24 years), *adult* (25–59 years), and *elder* (60+ years). Each neighborhood has its own unique *age* distribution, while the overall population consists of approximately 18.724% *children*, 15.706% *young*, 50.507% *adults*, and 15.061% *elders*.

For each neighborhood, a blob was initialized at the corresponding *home P*, with its total population determined based on the available neighborhood data and its sampled characteristics determined according to the previously described *age* and *occupation* distributions. Each blob was also assigned a traceable characteristic, *region of origin*, which indicates where the population resides and begins within the simulation. This traceable characteristic enables the identification of a blob’s origin throughout the simulation and ensures that the population returns to their respective residences after completing daily activities. Moreover, blobs originating from different regions cannot be merged, as their traceable characteristics remain distinct, preserving information about their place of origin. Appendix A.2 provides an example of the input data necessary for modeling an initial population in our simulations. For all experiments, a *cycle* was defined as 24 *cycle steps*, representing a full day and its hours, respectively. The following sections present results obtained from various simulation scenarios, illustrating the model’s capacity to capture population dynamics effectively.

4.2 Case Study 1: Random Walk

Gonzalez et al. [34] provide valuable insights into the dynamics of urban populations, which serve as a foundation for our abstracted urban mobility model. They highlight two significant observations: first, individuals tend to visit a select few locations repeatedly, and second, there is a certain level of randomness in their daily movements, akin to a Lévy flight random walk. A Lévy flight is a random walk pattern characterized by jumps of varying lengths, following a Lévy heavy-tailed probability distribution. In practical terms, this means that the Lévy flight predominantly consists of short jumps, with fewer long jumps.

To simulate this behavior, we have implemented a Lévy plugin, which introduces a new action type called the *levy walk*. Appendix A.4 presents an example of how new action types can be incorporated into our simulations using plugins. This action involves sampling distances from a Lévy distribution and moving a certain portion of

⁴Surveys conducted by the Observatory of the City of Porto Alegre (ObservaPOA). Data is available at: <http://www.observapoa.com.br/>

⁵Surveys conducted by the Empresa Pública de Tecnologia da Informação e Comunicação da Prefeitura de Porto Alegre (PROCempa). Data is available at: <https://portoalegremanalise.procempa.com.br/>

the population from a certain origin P_i to a destination P_j corresponding to the sampled distance. For each P , we calculate a histogram of distances to other P s, grouping them into bins based on a specified distance grouping value. Figure 7 illustrates the histogram of distances between P s, with each bin representing a range of 500 meters.

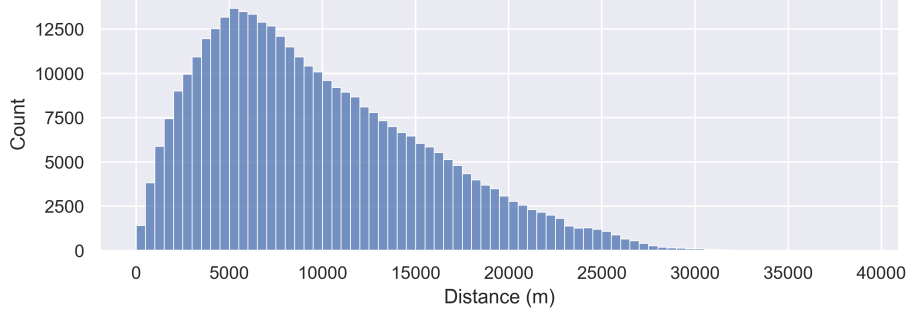


Fig. 7 Histogram of distances between points of interest (P s). Each bin represents a range of 500 meters.

When the *levy walk* action is triggered, the population at the origin P_i is uniformly divided into packets to be moved. Upon sampling a distance, the *levy walk* action randomly assigns a population packet to one of the P s within that distance group. Since the Lévy distribution is heavy-tailed (see Figure 8), it is possible to sample a distance that exceeds the farthest P , even if it is not very probable. In such cases, another distance is sampled from the distribution. Similarly, if there are no available P s within the sampled distance group, a new distance will be sampled.

A *levy walk* action A_i incorporates additional parameters $\mathbf{V}_i$ (Section 3.3) to enhance its functionality. These parameters include the scale parameter (*levy_scale*), which governs the Lévy distribution; the movement probability parameter (*levy_prob*), which determines the likelihood of a population packet being moved; the packet size parameter (*levy_pkt*), representing the group size that undergoes the random walk; and an optional label parameter (*levy_label*), enabling the selection of specific types of P s as potential destinations. Additionally, a *levy walk* action A_i requires a population template T_i to specify the desired population profile being affected. These parameters enable the simulation of random walk behavior across multiple scales: spatial scales, by adjusting the *levy_scale* parameter to match the simulated environment, and population scales, by modifying *levy_pkt*, which allows for the movement of large populations or individual entities (i.e., a blob with a total population of 1).

In our experiments, we design a global routine (*GRo*) that utilizes *levy walk* actions to disperse the working and student populations across the city throughout the day. Actions targeting workers relocate the population from a “home” P_h to a “work” P_w , utilizing a population template with criteria such as *Occupation* = *Worker* and *Age* = [*Young*, *Adult*, *Elder*]. These actions occur at each cycle step, with a higher movement probability (*levy_prob*) during the morning and afternoon (specifically,

steps 7 through 18). Actions targeting students relocate the population from a “home” P_h to a “school” P_s , using a population template with criteria such as $Occupation = Student$ and $Age = [Child, Young, Adult]$. These actions are executed only at specific cycle steps. Furthermore, the simulation routine includes *move population* actions in all “work” and “school” P s, returning a portion of their current population to their respective “home” P of each blob they contain. In both cases, the *region of origin* traceable characteristic of each blob allows us to identify the “home” P s where students and workers originate and return at the end of the day.

Subsequently, we conducted experiments by varying the *levy_scale* parameter in all *levy walk* actions to assess the population movements throughout the simulation. We define “displacement” as the movement of an individual from one P_i to another P_j , as part of the levy plugin. In the case of a blob B_i with a total population $TP_i = 250$, each movement between points of interest accounts for 250 displacements. All simulations maintain a population packet size of *levy_pkt* = 250, and the P s are grouped based on the distance to an origin P , with intervals of 500 meters, as depicted in Figure 7.

The experimental results are illustrated in Figure 8. All simulations consider a total population of 1,409,351. The x-axis represents the distance traveled during each displacement, grouped into 500-meter intervals. The y-axis indicates the total number of displacements that occurred at each levy configuration. The graph demonstrates that different values of the *levy_scale* parameter produce distinct distributions, where higher values yield more widely dispersed distributions of displacements, as observed in a simulation where *levy_scale* = 10.

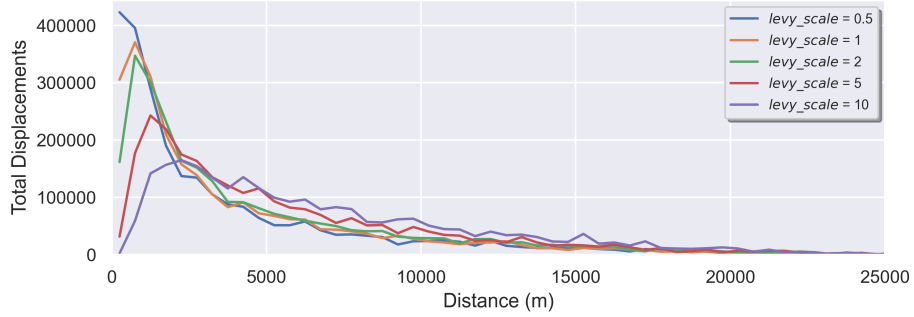


Fig. 8 Population displacement experiments. The x-axis represents the distance traveled during a displacement and is grouped in ranges of 500 meters. The y-axis represents the total number of displacements that traveled a certain distance. All simulations consider a total population of 1,409,351; a population packet size of *levy_pkt* = 250, which represents the group size that undergoes the random walk; and P s grouped by distance in ranges of 500 meters, as presented in Figure 7.

For the remaining experiments in this work, we adopt a scale parameter of *levy_scale* = 2 (represented by the green line in Figure 8). This particular value was determined empirically, as approximately 50% of the displacements observed in simulations using this parameter occurred within a 2km radius of the origin P .

Such displacements align with typical daily movements within a single neighborhood or between neighboring neighborhoods. However, this value can be adjusted to suit different environments, depending on the needs of the simulation designers.

Furthermore, we introduce an additional parameter, named *Restriction*, for the *levy walk* action to model movement restrictions. This parameter acts as a factor that reduces the total population being moved by each *levy walk* action, e.g., an *Restriction* = 0.25 would cause a *levy walk* action to move only $\approx 75\%$ of the population it would without the restriction. It allows for the simulation of varying population flows on different days of the week (e.g., weekends with different flow patterns than weekdays) or the simulation of population lockdown during a pandemic scenario. Figure 9 illustrates the results obtained by conducting a baseline experiment (without any restriction) and three other experiments involving different levels of movement restrictions. As previously mentioned, all simulations consider a scale parameter of *levy_scale* = 2.

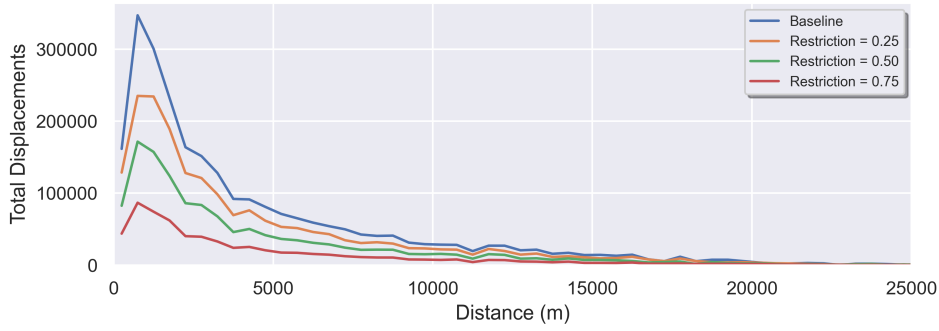


Fig. 9 Simulation of Movement Restriction with the *levy walk* action. All simulations consider a scale parameter of *levy_scale* = 2. The *Restriction* parameter acts as a factor that reduces the total population being moved by each *levy walk* action.

4.3 Case Study 2: Large-Scale Events

We also investigate the ability to simulate occasional events that disrupt the population’s daily routines, such as sports matches or musical concerts that occur on a weekly or monthly basis. To evaluate the impact of these large-scale events on mobility patterns, we compare their effects against a baseline scenario with no events. Furthermore, we examine how multiple concurrent events compete for the available population, highlighting the dynamics of population distribution.

To simulate these events, we implemented an additional plugin, which introduces a new action type called *gather population*. This action involves gathering the population from nearby P_s and relocating them to a designated destination P_d . A *gather population* action A_j considers a *gather_quantity* additional parameter (in V_j) to describe the total population to be relocated towards the destination P_d . The population quantity to be moved from each nearby P is determined by the population

available at the origin P and the distance to the destination. Once the population quantity that should be moved from each origin P is determined, the *gather population* action is further broken down into multiple *move population* actions. Each of these *move population* actions is responsible for transferring the population from an origin P to the destination P_d , ensuring that each origin P contributes to the movement.

For these experiments, we consider the routines of workers and students as a baseline simulation, utilizing a scale parameter of $levy_scale = 2$ as described in Section 4.2. Furthermore, we incorporate four large-scale events into the simulation, each occurring at a specific cycle step. These events are characterized by a *gather population* action, which requests the movement of 500,000 individuals to a designated destination P . The details of the events are specified as follows, considering the regions illustrated in Figure 6:

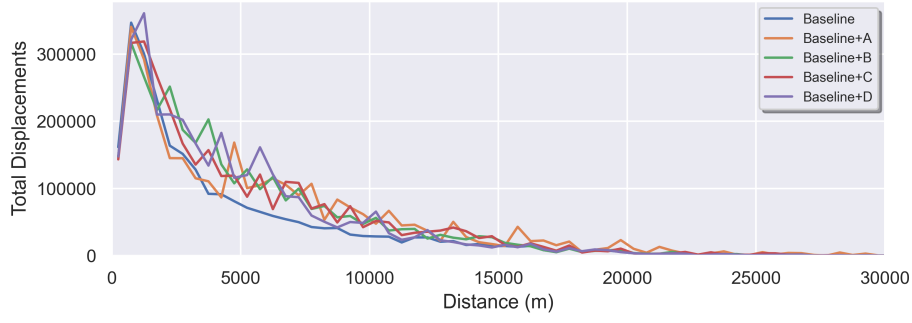
- Event (A): A sports event taking place at the Farrapos region stadium;
- Event (B): A sports event happening at the Praia de Belas region stadium;
- Event (C): A commercial event held at the marketplace in the Centro Histórico region; and
- Event (D): A cultural event occurring at our University in the Partenon region.

All events are scheduled to occur at cycle step 19, i.e., at 7:00 PM. Given that the city’s total population is 1,409,351, and the events require a total of 2,000,000 individuals, the events will compete for the available population if they all occur simultaneously in a single simulation. In addition, we model a scenario where some large-scale events take place at different cycle steps. In this alternate scenario, event C occurs at cycle step 9 (i.e., 9:00 AM), while event D occurs at cycle step 14 (i.e., 2:00 PM).

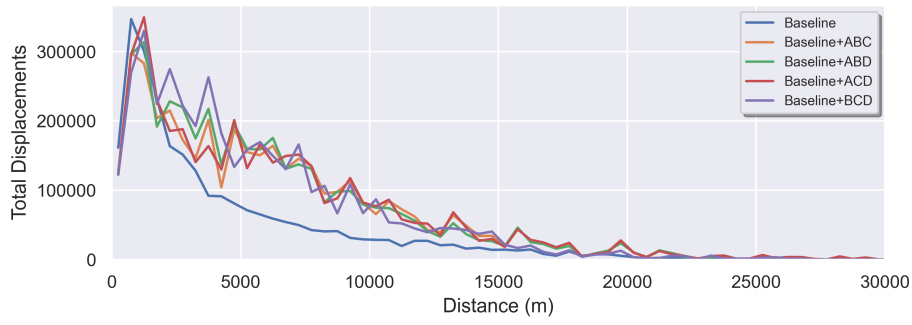
Table 4 provides an overview of the simulation results, including the total population displacements, the average displacement per city inhabitant, and the proportion of total displacements compared to the baseline simulation. Baseline means the “normal life” of the city, and the different scenarios in the table indicate the population displacements as a function of the existence of the events. For instance, the “Baseline+AB” scenario considers the Baseline in addition to Event A and Event B. In the two last lines of Table 4, we show the results of the Baseline with the existence of the four large-scale events occurring at the same time and with some occurring at different cycle steps.

Figure 10 presents the results of the simulation of large-scale events. Figure 10a visualizes the results of five simulations: the Baseline simulation (no events), a simulation with only Event A, a simulation with only Event B, only Event C, and only Event D. As indicated by the data in Table 4, simulations where only a single large-scale event occurs yield a similar number of total displacements. However, the different locations where each event occurred (i.e., different destination P s) and the available population nearby cause each simulation to have other total displacement counts for different distances. For example, the “Baseline+A” simulation (shown in orange in Figure 10a) presents an increased total displacement count near the 20,000m distance.

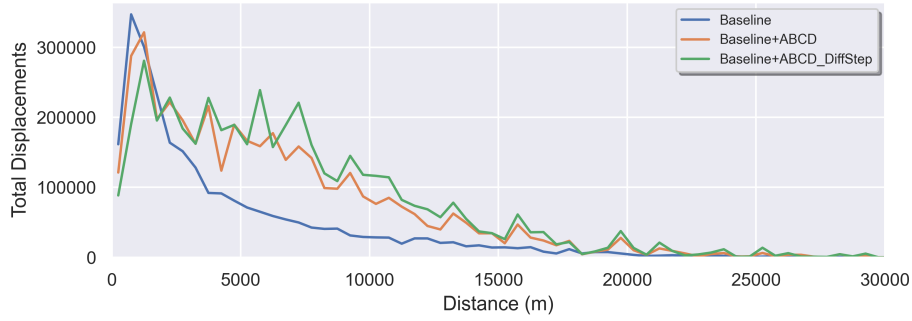
Figure 10b illustrates the results of five simulations, starting with the baseline simulation and followed by simulations with three events occurring at the same cycle



(a) Experiments include the Baseline scenario (no events), a scenario with only Event A, only Event B, only Event C, and only Event D.



(b) Experiments include the Baseline scenario and four other scenarios with three events occurring at the same cycle step (i.e., only one event missing in each).



(c) Experiments include the Baseline scenario, the Baseline in addition to all four events occurring at the same cycle step, and the Baseline in addition to all four events with some occurring at different cycle steps (DiffStep).

Fig. 10 Simulation of large-scale events.

Table 4 Results of the large-scale event simulations. Data includes the total population displacements, the average displacement per inhabitant of the city, and a proportion of total displacements compared to the baseline simulation. The “Baseline+AB” scenario, for example, considers the Baseline in addition to Event A and Event B.

Experiment Name	Total Displacements	Average Displacements per inhabitant	Increase over Baseline
Baseline	2,570,658	1.8240	—
Baseline+A	3,290,469	2.3347	28.00%
Baseline+B	3,300,782	2.3421	28.40%
Baseline+C	3,289,002	2.3337	27.94%
Baseline+D	3,281,102	2.3281	27.64%
Baseline+AB	3,786,856	2.6870	47.31%
Baseline+AC	3,748,417	2.6597	45.82%
Baseline+AD	3,804,017	2.6991	47.98%
Baseline+BC	3,738,497	2.6526	45.43%
Baseline+BD	3,839,126	2.7240	49.34%
Baseline+CD	3,826,094	2.7148	48.84%
Baseline+ABC	4,042,413	2.8683	57.25%
Baseline+ABD	4,109,929	2.9162	59.88%
Baseline+ACD	4,069,223	2.8873	58.29%
Baseline+BCD	4,081,563	2.8961	58.78%
Baseline+ABCD	4,235,926	3.0056	64.78%
Baseline+ABCD+DiffStep	4,638,329	3.2911	80.43%

step (i.e., only one event missing in each). As expected, and as indicated by the data in Table 4, the inclusion of additional events led to an increase in total displacements compared to the baseline simulation. However, it is important to note that although each event requests the movement of 500,000 individuals, the total displacements of three events do not increase by 1,500,000 compared to the baseline simulation. This is because individuals can move directly from their respective origin P_s (such as work or school) to the event P_s . Thus, workers can travel from their “work” P_s to the event directly, and students can travel from their “school” P_s to the event without the need to return to their “homes” before going to the events. Moreover, Events A, B, and C separately contribute to $\approx 730,000$ additional displacements to the baseline simulation, as shown in “Baseline+A”, “Baseline+B”, and “Baseline+C” in Table 4, respectively. In simulations where the three events occur simultaneously (i.e., “Baseline+ABC”), the events will compete for the available population in nearby neighborhoods, resulting in a reduced total population attending each event. Such result is presented in the last column of Table 4, where the number of displacements increased over the baseline is 57.25% for simulation “Baseline+ABC”. This is a lower amount than the sum of increased displacements of simulations “Baseline+A”, “Baseline+B”, and “Baseline+C”, which were 28.00%, 28.40%, and 27.94%, respectively.

Figure 10c presents the results of three simulations: the baseline simulation, a simulation with all four events occurring simultaneously, and a simulation with some events taking place at different cycle steps. As demonstrated by the data in Table 4, the simulation where events occurred at different cycle steps exhibited a significantly

higher number of displacements than the simulation with all four events happening simultaneously. This increase in the number of displacements can be attributed to two factors. Firstly, there is reduced competition for the available population when events are staggered. Secondly, individuals have the opportunity to attend multiple events throughout the day, increasing their individual displacements.

4.4 Case Study 3: Epidemic Contagion and Vaccination

Next, we explore our model’s ability to simulate behaviors observed over extended temporal scales, focusing on their long-term effects on the population. Specifically, we model the spread of an infectious disease as the population moves throughout the city, analyzing its progression over multiple simulation cycles, which represent consecutive days. Additionally, we evaluate the impact of public policies, such as vaccination campaigns and population lockdowns, on mitigating the disease’s spread.

To achieve this, we have implemented an additional plugin that introduces an *infection* action type. This action is based on the SIR model [51], which classifies individuals into one of three states: Susceptible, Infected, or Removed. The “Removed” state includes individuals who can no longer contribute to further contagion, either due to acquiring immunity (recovered) or death. The numerical dynamics of the SIR model are defined by Equations 2, 3, and 4.

$$\frac{dS}{dt} = -\frac{\beta IS}{N}, \quad (2)$$

$$\frac{dI}{dt} = \frac{\beta IS}{N} - \gamma I, \quad (3)$$

$$\frac{dR}{dt} = \gamma I, \quad (4)$$

where, $\frac{dS}{dt}$, $\frac{dI}{dt}$ and $\frac{dR}{dt}$ represent the rate of change over time for the susceptible, infected, and recovered populations, respectively, in a total population of N individuals. The parameters β and γ represent the infection and removal rates, respectively. The basic reproduction ratio, denoted as R_0 , is defined as $R_0 = \frac{\beta}{\gamma}$. It is important to note that the total population N remains constant in the SIR model, maintaining equality throughout the simulation.

To monitor the progression of the infection within the population, we introduced a traceable characteristic called *SIR Status* for all blobs in the simulations. This characteristic can take one of three possible values: “Susceptible”, “Infected”, or “Removed”. As outlined in Section 3.2, each blob in the simulation consists of individuals who share the same *SIR Status* value. However, blobs with different *SIR Status* values can coexist within the same P , allowing for the simultaneous representation of susceptible, infected, and removed individuals at a given location.

In the context of epidemic simulations, we utilize the routines of workers and students as the baseline, employing a scale parameter of $levy_scale = 2$ as described in Section 4.2. Additionally, we incorporate the *infection* action into the global routine *GRo*, which occurs at every cycle step. When triggered, the *infection* action identifies the current population with each SIR status in a given P and applies Equations 2, 3,

and 4 based on the β and γ parameters. We conducted experiments considering different values for the basic reproduction ratio (R_0), as this value can vary across different countries and cities. We specifically selected R_0 values of 2.6, 3.07, and 6.0 based on research conducted on Brazil’s COVID-19 data [52–54]⁶. For all experiments, we set $\gamma = \frac{1}{15}$, indicating that individuals transition from the “Infected” to the “Removed” state over a 15-day period. The values of β are defined as 0.1733, 0.204, and 0.4, respectively, to correspond with the R_0 values of 2.6, 3.07, and 6.0.

As part of the simulation setup, we empirically determined that each region of the environment begins with 50 randomly selected individuals classified as “Infected,” 0 individuals classified as “Removed,” and the remaining population classified as “Susceptible.” This equates to a total of 4,700 individuals initially categorized as “Infected” across all regions. The simulations were run for a period of 500 cycles, where each cycle consists of 24 cycle steps representing days and hours of the day, respectively.

Figure 11 illustrates the results of the simulations conducted with different basic reproduction ratios (R_0). The x-axis denotes the simulation cycle (i.e., day), while the y-axis shows the cumulative number of infected individuals up to that point, including those who transitioned from the “Susceptible” to the “Infected” state, as well as the initial infected population (4700). As depicted, higher values of R_0 directly influence the speed of infection spread and the cumulative number of infected individuals throughout the simulation.

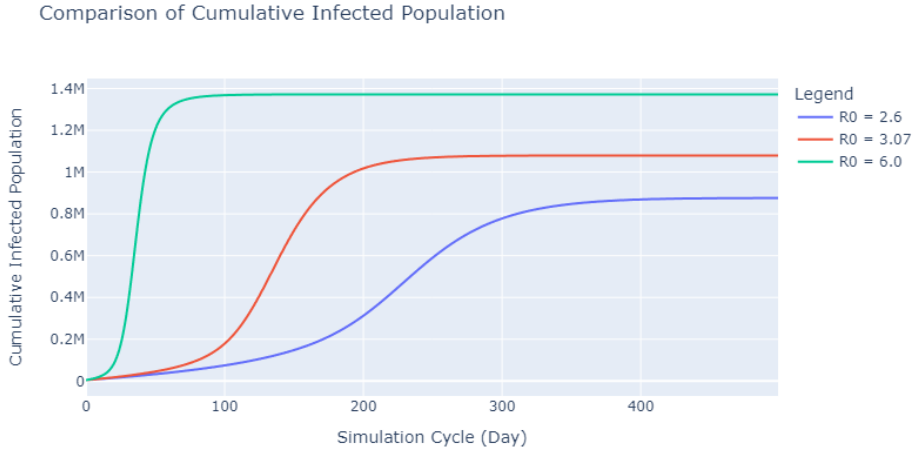


Fig. 11 Simulations conducted with different basic reproduction ratios (R_0). The x-axis represents the simulation cycle (i.e., day), while the y-axis represents the cumulative infected population up to that point in the simulation.

For the remaining experiments, we will consider a baseline value of $R_0 = 2.6$ for the simulations. Figure 12 displays the total population amount for each possible value

⁶Additional research on R_0 on Brazilian state capitals available [HERE](#)

of the “SIR Status” traceable characteristic over time in a simulation with $R_0 = 2.6$. It can be observed that despite reaching a cumulative infected population of 875,962 people (as shown in Figure 11), the peak of the infection occurred on day 247, with a total of 81,528 infected individuals at that point.

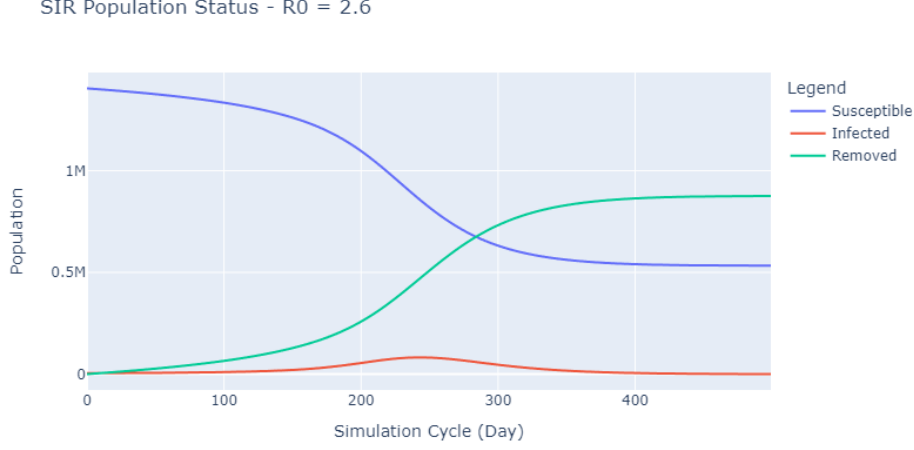


Fig. 12 Population count over time for each “SIR Status” traceable property value in a simulation with $R_0 = 2.6$

Next, we extend the simulation to include additional behaviors related to epidemic contagion, specifically population vaccination as a measure to mitigate the spread of the disease. To facilitate this, we introduce a traceable characteristic called *vaccine level* to all blobs in the simulation. This property is represented by an integer value. Additionally, we implement a new action called *vaccinate*. When this action is triggered, a portion of the population is moved to a nearby “pharmacy” P , which is available in all regions of the environment. At the pharmacies, the population receives a dose of the vaccine, resulting in an increase of their *vaccine level* by 1. This signifies that they have been vaccinated and their resistance to the transmission of the virus has improved. Subsequently, the newly vaccinated population returns to their respective P s, such as workers returning to their “work” P .

In our simulations, we explore various vaccination configurations by considering different dose counts, which represent the number of doses available to the population, as well as the effectiveness of each dose. Table 5 summarizes the vaccination settings used in our experiments, where the Dose Count ranges from 1 to 3. The Effectiveness parameter indicates the efficacy of each dose in mitigating the spread of the infection.

All blobs in simulations with vaccinations contain a *vaccine level* corresponding to an Effectiveness value in Table 5. Whenever a blob with *SIR Status* = *Susceptible* gets infected, the amount of people that should be infected is defined by Equation 3. The Effectiveness works by multiplying the result of Equation 3 by $1 - Effectiveness$,

i.e., an *Effectiveness* = 0.4 reduces the amount of people changing from “Susceptible” to “Infected” by 40%. Starting Day indicates the simulation cycle at which each dose begins to be administered to the population. It is important to note that the second and third doses, if available, can only be administered to individuals who have already received the previous dose. Configurations A through C represent increasing effectiveness as more doses are administered, while Configuration D represents an ideal vaccine with 100% effectiveness in preventing infection.

Building upon the parameters defined in previous experiments, all vaccination experiments utilize the routines of workers and students as the baseline, employing a scale parameter of *levy_scale* = 2 and an *infect* action with $R_0 = 2.6$. The global routine *GRO* also includes the *vaccinate* action, performed at selected cycle steps during the morning and afternoon of each day. To determine the number of individuals to be moved to “pharmacy” *Ps* on each simulation day, we gathered data on the daily number of COVID-19 vaccines administered in Porto Alegre⁷. This data is then proportionally distributed among the regions in the environment based on their respective populations.

Table 5 Configuration of vaccines for experiments. Dose Count represents the number of doses available for the population. Effectiveness represents how effective each dose is in reducing the spread of the infection. Starting Day represents the day (i.e., simulation cycle) when each dose starts being administered to the population. The second and third doses can only be applied to people who already received the previous dose. While VaccinationA, B, and C state for the doses, VaccinationD simulates a scenario where the effectiveness of the vaccine is 100%.

Vaccination Config	Dose Count	Effectiveness	Starting Day
VaccinationA	1	[0.4]	[50]
VaccinationB	2	[0.4, 0.7]	[50, 150]
VaccinationC	3	[0.4, 0.7, 1.0]	[50, 150, 250]
VaccinationD	1	[1.0]	[50]

Figure 13 showcases the results of the vaccination simulations. Despite the “VaccinationA” configuration having an effectiveness of only 0.4, a significant impact on the cumulative infected population can be observed compared to the baseline. The baseline simulation resulted in a total of 875,962 infected individuals, whereas the “VaccinationA” configuration achieved a notably lower count of 440,569 infected individuals. The incorporation of additional doses in the “VaccinationB” and “VaccinationC” configurations further decreased the cumulative infected population, resulting in 360,607 and 355,124 infected individuals, respectively. Although “VaccinationC” contains an effectiveness of 1.0, this efficacy is only realized when the population undergoes the third dosage, initiated on day 250 of the simulation. Consequently, “VaccinationB” and “VaccinationC” are very similar until day 250, at which point the third dosage of “VaccinationC” begins to exert its influence, resulting in a small difference in the

⁷Dashboard of Brazilian vaccination data available [HERE](#) (text in Brazilian Portuguese)

cumulative infected population of both simulations. Notably, the “VaccinationD” configuration, representing an ideal vaccine, demonstrates the lowest number of infections, with only 170,978 individuals affected.

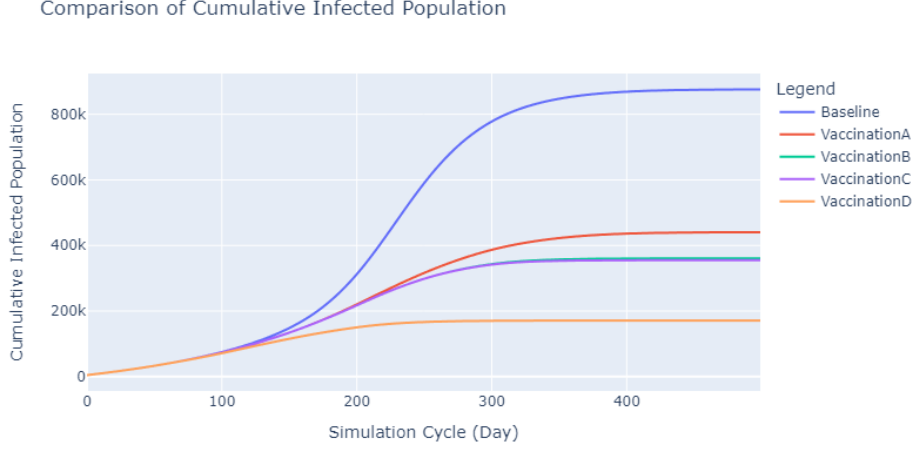


Fig. 13 Simulations of different vaccination configurations, as described in Table 5. All simulations consider a basic reproduction rate of $R_0 = 2.6$.

Finally, we explore the incorporation of movement restrictions in our epidemic contagion experiments to simulate lockdown behavior. The movement restriction is described in Section 4.2 and illustrated in Figure 9. These experiments encompass a combination of three movement restriction values (no restriction, 0.5, and 0.75) that represent percentages of the population who do not leave their living neighborhoods, along with two vaccination scenarios: no vaccination; or “VaccinationC”, which includes three doses. We chose to compare the “VaccinationC” configuration rather than “VaccinationD” because “VaccinationC” exhibits a more realistic behavior. In “VaccinationC”, the effectiveness of the vaccine increases with the number of doses a person receives. In contrast, “VaccinationD” represents an idealized vaccine with a fixed effectiveness of 1.0 achieved in a single dose.

Figure 14 illustrates the results obtained from our simulations. The imposition of movement restrictions reduces the cumulative infected population compared to the baseline scenario (no restriction and no vaccination). However, including population vaccination (experiment “VaccC”) exhibits more significant effectiveness in reducing the infected population compared to simulations with only movement restrictions. Moreover, the combination of movement restriction and vaccination demonstrates the most favorable outcomes, with the simulation containing a movement restriction of 0.75 and vaccination resulting in only 140,941 infected individuals.

Comparison of Cumulative Infected Population

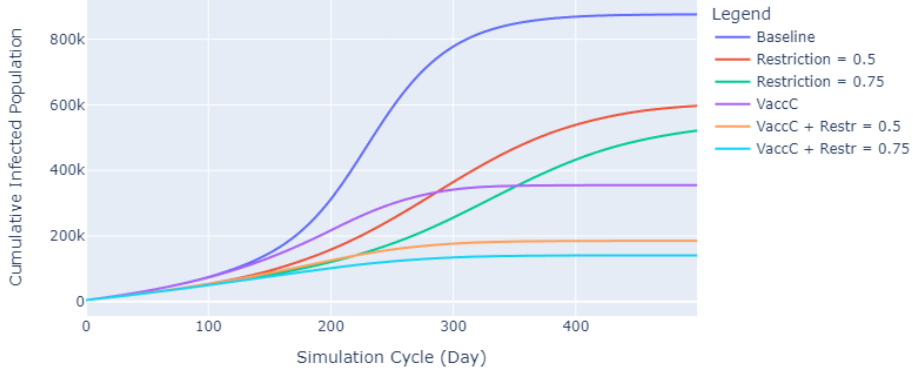


Fig. 14 Simulations of epidemic contagion. Experiments include a combination of three movement restriction values (no restriction, 0.5 and 0.75) to represent a lockdown and two vaccination scenarios (no vaccination or “VaccinationC”). All simulations consider a basic reproduction rate of $R_0 = 2.6$.

4.5 Quantitative Performance Evaluation

To complement the qualitative case studies, we present a quantitative evaluation of LodusPop’s computational efficiency. All benchmarks use the Porto Alegre city model described in Section 4.1 (94 regions, 504 points of interest, and a total population of 1,409,351). Each configuration was executed five times, and average values are reported.

Table 6 reports quantitative information for each *Case Study* and *Configuration*, as follows: (i) *Max Blob Count*, the maximum number of blobs observed during the run; (ii) *Number of Cycles*, where a cycle corresponds to one simulated day in our setup, with 24 cycle steps; (iii) *Avg Sim Time (s)*, the average total real-time required to complete the run; (iv) *Time Per Cycle (s)*, the average runtime per simulated day; (v) *MemProfile Avg Peak (MiB)*, the average peak resident memory of the Python process⁸; and (vi) *TraceMalloc Avg Peak (MiB)*, the average peak traced Python allocations⁹. We interpret *Max Blob Count* as an indicator of simulator workload: more frequent splits increase the number of population units to process and reduce merge opportunities, typically raising both runtime and memory consumption.

Using the random walk configuration of *levy_scale = 2* (Section 4.2) as the baseline simulation for the remaining case studies and configurations, the simulator completes a full run in 5.211s on average, with *MemProfile Avg Peak* of 234.9 MiB, and a *TraceMalloc Avg Peak* of 58.565 MiB. Introducing movement restrictions (Section 4.2) slightly reduces *Time Per Cycle* (down to 4.795s at *Restriction = 0.75*),

⁸ Calculated with Memory Profiler v0.61.0: <https://pypi.org/project/memory-profiler/>

⁹ Calculated with TraceMalloc: <https://docs.python.org/3/library/tracemalloc.html>

while peak memory remains essentially unchanged. This indicates that reduced mobility mainly decreases the number of executed move operations, without substantially changing blob splitting, as expected.

Large-scale events (Section 4.3) increase runtime because they introduce additional gathering actions that relocate populations across distant locations. *Time Per Cycle* rises from 6.144s (average simulations with one event) to 12.172s (simulation with four simultaneous events). These scenarios tend to produce fewer blobs, as populations aggregate around event locations, yet *MemProfile Avg Peak* and *TraceMalloc Avg Peak* remain close to the baseline (≈ 235 MiB and ≈ 59 MiB, respectively). This indicates that the dominant cost comes from executing the large-scale gathering operations by pulling individuals from multiple regions and points of interest, rather than from an increase in blob count.

Epidemic contagion and vaccination simulations (Section 4.4) are more demanding because additional traceable characteristics (e.g., SIR status and vaccine level) increase population heterogeneity, causing more frequent splits and reducing merge opportunities. In our model, blobs can only merge when their traceable attributes match; as the simulation progresses, individuals become distributed across multiple states (susceptible, infected, removed) and vaccination levels, so blobs with different traceable characteristics cannot be merged back together. For infection-only experiments, 500-cycle runs require 9.899–11.654s per cycle, and *MemProfile Avg Peak* increases to roughly 1.72–1.80 GiB. Vaccination scenarios further intensify splitting: VaccinationC reaches a *Max Blob Count* of 23,355 and requires 33.455s per cycle, with *MemProfile Avg Peak* of 2.86 GiB. Overall, *Time Per Cycle* closely follows increases in *Max Blob Count*, reinforcing that split and merge behaviors are key components of computational cost.

These measurements reflect the simulator execution on a single machine and are therefore hardware-dependent. The results are intended as a quantitative evaluation of the presented case studies, providing direct support for the efficiency and scalability claims within their scope, rather than as a head-to-head comparison with other simulation frameworks. All experiments were run on a machine with an AMD Ryzen 5 5600X CPU, an NVIDIA GeForce RTX 2060 GPU, 16GB DDR4 2666MHz (CL19) RAM, and a KINGSTON SNV3S1000G SSD, using Python v3.13.6 on Microsoft Windows 11 Education (Version 10.0.262000).

4.6 Original Study Synthesis and Scope

The original evaluation demonstrates three forms of extensibility without changing the blob representation. The random-walk study adds a stochastic movement action, the large-event study temporarily changes routine demand, and the epidemic study changes traceable characteristics through contagion and vaccination policies. The quantitative evaluation then characterizes runtime, memory, and blob counts for these published configurations. Sections 4.1–4.6 preserve those results as originally reported.

The case scenario nevertheless has a deliberate spatial limitation. Porto Alegre is represented by 94 neighborhood regions, and each neighborhood contains one aggregated *home* point of interest holding its resident population. Residents are not distributed among census-sector home nodes. The original experiments also do not

Table 6 Quantitative performance evaluation of LodusPop. We use the Random Walk mobility configuration (Lévy-flight simulation with *levy_scale* = 2; Section 4.2) as the baseline reference for the remaining case studies and configurations. For each case study and configuration, we report the maximum blob count observed during execution, the number of simulated cycles (days), the average total simulation time over five runs, the corresponding time per cycle, and peak memory usage measured via MemProfile (process RSS) and TraceMalloc (Python allocation tracing).

Case Study	Configuration	Max Blob Count	Number of Cycles	Avg Sim Time (s)	Time Per Cycle (s)	MemProfile Avg Peak (MiB)	TraceMalloc Avg Peak (MiB)
Baseline (Random Walk) Case Study 1	<i>levy_scale</i> = 2	2857	1	5.211	5.211	234.912	58.565
Movement Restriction (Fig 9) Case Study 1	Restriction = 0.25	2871	1	5.074	5.074	234.957	58.577
	Restriction = 0.50	2864	1	4.958	4.958	234.935	58.527
	Restriction = 0.75	2883	1	4.795	4.795	235.234	58.516
Large-Scale Events (Fig 10 and Table 4) Case Study 2	Average of 1 Event	2671	1	6.144	6.144	235.355	58.767
	Average of 2 Events	2651	1	7.986	7.986	235.297	58.892
	Average of 3 Events	2651	1	9.910	9.910	235.530	59.061
	4 Events	2651	1	12.172	12.172	235.825	59.217
	4 Events at Different Times	2079	1	11.153	11.153	235.355	59.146
Infection (Fig 11) Case Study 3	$R_0 = 2.6$	8840	500	5826.840	11.654	1798.114	1554.886
	$R_0 = 3.07$	8830	500	5630.155	11.260	1741.606	1520.793
	$R_0 = 6.0$	8818	500	4949.313	9.899	1718.252	1480.354
Vaccination (Fig 13 and Table 5) Case Study 3	VaccA	16495	500	8991.828	17.984	2246.849	2057.299
	VaccB	20840	500	11346.181	22.692	2250.278	2318.228
	VaccC	23355	500	16727.571	33.455	2862.153	2512.703
	VaccD	12542	500	7620.026	15.240	1750.923	1883.745
Vaccination and Restriction (Fig 14) Case Study 3	$R_0 = 2.6$; Restriction = 0.50	9011	500	4008.347	8.017	1983.816	1743.626
	$R_0 = 2.6$; Restriction = 0.75	8320	500	4566.876	9.134	1681.857	1472.081
	VaccC; Restriction = 0.50	16722	500	9279.227	18.558	2628.414	2313.595
	VaccC; Restriction = 0.75	15429	500	8139.657	16.279	2540.851	2240.514

enforce facility capacities, disable locations dynamically, propagate dependencies, or test destination substitution. Consequently, they establish that the same core population abstraction supports several action types, but they do not evaluate access to scarce services or behavior under infrastructure disruption.

The following sections extend the evidence rather than replace these results. The recent scenarios distribute residents among census-sector homes, add hospitals, dialysis clinics, water-treatment facilities, work and school nodes, and activity points of interest, and evaluate repeated disruption and adaptation conditions. Their common design and provenance are introduced in Section 5. General implications of spatial abstraction are deferred to Section 9.3, while immersive and geographic-flow visualizations are discussed separately in Section 8.

5 Extended Evaluation Design and Data

5.1 Evaluation Questions and Scenario-Comparison Strategy

The recent experiments extend the original evaluation along two application strands. The health-service strand asks how capacity, routing, recovery time, spatial distribution, and supporting infrastructure affect access to inpatient and dialysis services. The mobility strand asks how the loss of homes and destinations affects routine activity and commuting and whether a destination-substitution policy restores the requested activity. These experiments share the LodusPop core but not a single artificial “master scenario.” Each family retains the time semantics, demand rules, capacities, and outcomes needed by its research question.

All conclusions are based on contrasts between modeled scenarios. A reference condition is paired with one or more controlled modifications, such as increased demand, an unavailable clinic, flood exposure, or enabled rerouting. This design supports statements about the effect of an assumption within the model. It does not by itself establish that the simulated demand, routing, or adaptation reproduces observed behavior.

5.2 Spatial Representations of Porto Alegre

Table 7 distinguishes the three principal spatial representations. The original published scenario uses one home per neighborhood. The recent inputs place homes at census-sector locations within regions, so several homes can coexist in a neighborhood. The 13-region environment is a reduced study area used for the flood matrix and reduced dialysis network, while the 94-region environment provides complete-city coverage for selected no-flood mobility, inpatient, and dialysis configurations.

Table 7 Spatial representations used in the thesis. Counts describe the stored environments used by the reported studies.

Representation	Regions	Home nodes	Purpose
Original neighborhood scenario	94	94	Published random-walk, large-event, epidemic, and performance experiments; one aggregated home per neighborhood.
Reduced census-sector scenario	13	435	Flood and adaptation matrix and reduced dialysis experiments; multiple home nodes per region.
Complete census-sector scenario	94	2,711	Complete-city inpatient and dialysis configurations and the no-flood Popular Times evaluation.

The finer representation changes scenario inputs and the distances from which populations may be selected; it does not alter the definitions of regions, points of interest, blobs, templates, or actions. Domain inputs add hospitals, dialysis clinics, water-treatment facilities, workplaces, schools, marketplaces, restaurants, and pharmacies as ordinary points of interest with type-specific attributes consumed by plugins.

5.3 Population, Facilities, and External Data

The census-sector population files allocate residents to home nodes and retain population characteristics used by movement plugins. The original and recent experiments therefore use the same population abstraction at different input granularities. Popular Times V2 combines initial home populations with fixed weekly hourly profiles for marketplaces, restaurants, and pharmacies. Levy Walk V2 allocates worker and student attendance targets across work and school destinations and divides movements into packets of at most 50 people.

The inpatient environment incorporates hospital locations and January 2024 bed-capacity data derived from the LODUS-Health CNES dataset [21]. Capacity is separated into public (SUS) and private pools. The dialysis inputs add clinics, opening schedules, treatment capacities, and water-treatment nodes. Demand magnitudes, recovery schedules, facility allocations, and routing mappings are experiment configuration, even when facility locations or capacities originate in an external dataset.

The flood experiments use the repository’s time-indexed water-level series and node exposure attributes. At each step, the flood plugin may add or remove a flood disable cause. This design keeps the environmental observation, the threshold that maps it to availability, and the movement response as separate layers.

5.4 Dynamic Availability, Flooding, and Dependencies

Direct availability changes cover scheduled closures, experiment-specific outages, and flood exposure. Every cause is tracked independently. Dependency rules are then evaluated using the effective states defined in Equation 1. In the dialysis scenarios, clinics can depend on water-treatment facilities selected through shared attributes. Disabling a required facility propagates a dependency blocker to the clinic; restoring one direct cause does not remove unrelated blockers.

Availability does not automatically move population. The dialysis plugin excludes disabled clinics from admission, the inpatient plugin can use an enabled fallback, Popular Times V2 can suppress or reroute visits, and Levy Walk V2 records unavailable origins, unavailable destinations, temporary returns, blocked returns, and repatriation. This division allows the same node-state mechanism to support different domain responses.

5.5 Replications, Matched Seeds, Metrics, and Validation

Table 8 summarizes the completed experiment batches. Matched seeds reduce unrelated stochastic variation when two configurations are compared. Deterministic configurations are run once and are not assigned artificial confidence intervals. Repeated stochastic conditions are summarized by their reported means and 95% intervals.

The mobility and dialysis studies use 24 hourly steps per daily cycle. Popular Times and Levy Walk V2 run for 56 days, whereas the dialysis catalog runs for 10 days. Inpatient care uses 12 abstract admission/discharge steps arranged as three cycles of four; those steps should not be interpreted as hours. Validation includes

Table 8 Recent experiment batches included in the thesis.

Batch	Scenarios	Runs	Principal outcomes
Inpatient care	7	210	Coverage, overflow, queue length, admission delay, specialty/payer service, and distance.
Dialysis resilience	12	361	Treatment coverage, overdue demand, waiting exposure, admission distance, and disabled clinic-hours; one extra K03 record is retained.
Popular Times V2 core	10	130	Visit fulfillment, travelers, distance, occupancy, population conservation, and node-state validity.
Destination adaptation	2	31	Rerouted and unmet visits, added displacement, receiving load, and fulfillment.
Levy Walk V2	5	150	Commute fulfillment, unmet reasons, moved population, distance, return lifecycle, and activity-visit fulfillment.

population conservation, enabled-node movement checks, exact cycle-level attendance where specified, demand accounting, and specialized integrity checks. All 130 Stage 4, 31 Stage 5, and 150 Stage 6 mobility runs passed their respective invariants; the inpatient package passed 2,052 integrity checks and five determinism checks.

5.6 Data Provenance and Modeling Assumptions

Observed, derived, and synthetic inputs are reported separately. Geographic locations, census-sector population inputs, hospital capacities, and the water-level series provide empirical context, subject to their collection dates and preprocessing. Visit rates, demand profiles, outage schedules, thresholds, recovery times, synthetic facilities, CID mappings, and some routing allocations are model configurations. In particular, the synthetic Santa Ana capacity and the CID-based inpatient demonstration are not observations of clinical practice.

Stochastic repetitions quantify variability produced by the implemented sampling process, not uncertainty in every input parameter. Some demand components are deterministic and therefore produce identical fulfillment values across seeds. External validity requires calibration and comparison with observed mobility or service data and is discussed in Section 9.6.

The numerical tables in Sections 6 and 7 are transcribed from the preserved repository summaries: `results/dialysis/analysis/scenario_summary.csv`, `results/inpatient_care/analysis/report.md`, `docs/results/popular_times_v2_stage4/REPORT.md`, `docs/results/popular_times_v2_stage5/REPORT.md`, and `output_logs/popular_times_v2_stage6/REPORT.md`. Raw run directories and metadata remain the audit sources when an alternative aggregation is required.

6 Urban Health-Service Resilience

6.1 From LODUS-Health to the Recent Experiments

LODUS-Health introduced a Porto Alegre hospital dataset, a configurable bed-occupancy simulation, a web interface, and medical-text generation [21]. It provides

the conceptual and data basis for the recent inpatient work. The present evaluation does not merge its original figures or simulation outcomes with the new batch. Instead, the January 2024 capacity source and the idea of moving cohorts from homes to hospitals are carried forward into a new plugin with explicit payer pools, lengths of stay, overflow and queue policies, and auditable demand records.

The dialysis study uses the same general pattern—population demand traveling from homes to geographically located services—but introduces recurring treatment, clinic schedules, admission capacity, recovery, flooding, and infrastructure dependencies. The two studies therefore test related access questions while retaining distinct service semantics and metrics.

6.2 Inpatient-Care Model and Scenario Design

Inpatient demand is represented by cohorts sharing an origin, diagnosis or specialty category, payer, preferred destination, and length of stay. Each hospital exposes separate SUS and private capacity pools. Discharges release capacity after the configured stay. A disabled preferred hospital may be replaced by an enabled configured fallback, but the experiments do not assume unconstrained dynamic load balancing.

The *overflow* policy admits all eligible demand and records occupancy beyond the configured bed count. It is a counterfactual for measuring latent shortage, not an operational policy. The *queue* policy enforces capacity and retains unserved cohorts in a first-in, first-out queue. Seven scenarios cover reference and high demand under both policies, a fixed two-facility allocation under both policies, and a small queue-based CID-routing demonstration. Each scenario uses seeds 0–29.

6.3 Inpatient-Care Results

Table 9 reports the batch summary. Reference demand is almost fully served under the queue policy. Under high demand, unconstrained overflow reveals an average peak excess of 333.8 patients, while enforcing capacity reduces mean coverage to 52.10% and leaves an average final queue of 1,336.5. The fixed two-facility configuration raises queue-policy coverage to 97.08% and reduces the final queue to 77.2 patients. The CID demonstration achieves 96.13% mean coverage and a population-weighted mean admission distance of 3.16 km.

The two-facility result demonstrates the consequence of a configured spatial allocation, not an inferred optimal policy. Santa Ana’s psychiatric capacity is synthetic. The CID demand and mappings are also synthetic, and the six-origin demonstration is too small to support clinical conclusions. Patients admitted or waiting at the final step are horizon-censored, and the original per-step distributions from the earlier dissertation experiments were not retained; exact reproduction of those earlier timings is therefore outside the scope of this batch.

6.4 Dialysis-Service Model and Scenario Design

The dialysis plugin creates patients with recurring treatment demand every three days, assigns them to home nodes, and admits due patients to enabled clinics subject to schedules and per-step capacity. Unserved patients remain due, allowing treatment

Table 9 Inpatient-care outcomes over 30 seeds. Intervals are the reported P05–P95 ranges.

Scenario	Coverage	Peak overflow	Final queue	Mean delay
Reference overflow	1.0000 [1.0000–1.0000]	0.4 [0.0–2.5]	0.0	0.00
Reference queue	0.9996 [0.9972–1.0000]	0.0	0.2 [0.0–1.5]	0.00
High-demand overflow	1.0000 [1.0000–1.0000]	333.8 [320.4–349.1]	0.0	0.00
High-demand queue	0.5210 [0.5105–0.5297]	0.0	1,336.5 [1,312.0–1,365.8]	2.44 [2.34–2.56]
Two-facility overflow	1.0000 [1.0000–1.0000]	52.5 [37.5–70.0]	0.0	0.00
Two-facility queue	0.9708 [0.9570–0.9809]	0.0	77.2 [50.5–113.5]	0.14 [0.09–0.21]
CID-routing queue	0.9613 [0.9151–1.0000]	0.0	8.7 [0.0–19.1]	1.14 [1.02–1.26]

coverage, waiting exposure, and overdue demand to be measured. Admission events retain the home origin, receiving clinic, and distance.

The K00–K11 catalog compares a reduced 13-region service network with a complete-city network. Conditions include reference and moderate demand, above-capacity demand, a persistent outage, rapid recovery after four hours, a historical clinic-flood pattern, a synthetic water-treatment failure, combined disruptions, and two failure topologies with matched aggregate capacity loss. Disruptions begin on the third simulated day, and the primary comparisons use matched seeds 0–29.

6.5 Dialysis-Service Results

Reference and moderate-demand scenarios K00, K01, and K10 complete every due treatment. Above-capacity K07 attains 93.02% coverage despite no facility outage. The persistent Moinhos outage in K02 reduces coverage to 68.52%, whereas four-hour recovery in K03 restores 100% coverage. The historical clinic-flood condition K04 also maintains complete coverage through available alternatives.

The synthetic water-treatment failure in K05 and the combined reduced-network disruption in K06 each reduce coverage to 50%. In the complete network, the corresponding combined disruption K11 maintains 71.43% coverage. Thus, a larger network improves the modeled outcome but does not eliminate the effect of widespread dependent failures.

Failure topology changes travel even when coverage and nominal lost capacity are similar. K08 removes one high-capacity clinic and K09 removes several lower-capacity clinics; both maintain complete coverage, but mean admission distance rises from 1.768 km in K08 to 2.037 km in K09. The comparison indicates that aggregate capacity alone does not characterize the spatial burden imposed by a disruption.

6.6 Health-Service Synthesis

The two health studies expose complementary aspects of access. Inpatient care shows the difference between latent shortage and an enforced queue, and it shows how a configured distribution of demand across facilities can change coverage and delay.

Table 10 Mean dialysis outcomes. Each scenario has 30 primary seeds; K03 retains one additional completed record.

ID	Condition	Coverage	Distance (km)	Interpretive role
K00	Reduced reference	1.0000	1.716	Reference demand
K01	Reduced moderate	1.0000	1.836	Disruption control
K02	Persistent Moinhos outage	0.6852	2.090	Persistent isolated failure
K03	Four-hour recovery	1.0000	1.833	Recovery contrast
K04	Historical clinic flood	1.0000	1.768	Clinic exposure
K05	Synthetic water failure	0.5000	1.845	Cascading dependency
K06	Combined reduced failure	0.5000	1.845	Compound disruption
K07	Reduced above capacity	0.9302	1.854	Demand stress
K08	One high-capacity failure	1.0000	1.768	Concentrated topology
K09	Matched small failures	1.0000	2.037	Distributed topology
K10	Complete moderate	1.0000	6.537	Complete-network control
K11	Complete combined	0.7143	7.052	Complete-network disruption

Dialysis shows how recurring service, recovery time, failure topology, and supporting infrastructure affect treatment completion and travel. Across both, total capacity is necessary but insufficient: where capacity is located, whether it is available, and how demand is routed determine the modeled burden. Coverage, overflow, queues, overdue treatments, and distance remain domain-specific measures and are not collapsed into a single resilience score.

7 Routine Mobility, Flood Disruption, and Adaptation

7.1 Census-Sector Mobility Scenario

Popular Times V2 models weekly hourly visits to marketplaces, restaurants, and pharmacies. Demand is derived from the initial population associated with each home node and fixed weekly profiles. Travelers are sourced from enabled homes, occupy a destination for one hour, and then return to their prior node. The reduced environment contains 13 regions and 435 census-sector homes; the complete environment contains 94 regions and 2,711 homes.

The flood schedule can affect homes, activity points of interest, or both. A disabled home is not used as a source, while a disabled destination either suppresses its request or, in the adaptation condition, redirects it to the nearest enabled point of interest of the same type. Legacy Levy movement can be enabled alongside these visits to change the population distribution from which travelers are sourced.

7.2 Popular Times V2 Core Matrix

The Stage 4 matrix contains 130 runs over 56 simulated days. Levy-off configurations are deterministic and run once. The 13-region Levy-on configurations use matched seeds 0–29, and the 94-region Levy-on no-flood configuration uses five seeds. All runs pass population-conservation and enabled-node movement checks.

Flooding activity destinations suppresses 83,445 requested visits, reducing fulfillment from 100% to 96.49%. Flooding homes alone maintains complete visit fulfillment

but increases sourcing distance because travelers must be drawn from other enabled homes. Enabling legacy Levy movement changes sourcing distance but not the amount of requested activity fulfilled. Table 11 reports the principal 13-region Levy-on conditions. The 94-region comparison is limited to the no-flood baseline and also achieves complete fulfillment.

Table 11 Stage 4 13-region Levy-on outcomes over 30 matched seeds.

Flood exposure	Fulfillment	Travelers/capita	Distance/request	Interpretation
None	1.0000	17.9997	81.9953	Reference
POIs	0.9649	17.3672	76.2669	Destination demand suppressed
Homes	1.0000	17.9997	88.9408	Full demand sourced farther away
Homes and POIs	0.9649	17.3672	76.8287	POI loss determines fulfillment

7.3 Destination Adaptation

Stage 5 evaluates nearest-enabled, same-type POI substitution for the combined home-and-POI flood condition. The deterministic Levy-off run and 30 matched Levy-on runs all pass their invariants. In every run, all 83,445 visits previously suppressed by a disabled destination are rerouted, restoring fulfillment from 96.49% to 100%. Mean additional POI displacement is 297.89 m per rerouted traveler, and the maximum receiving load is 92 travelers. These values measure the cost and concentration produced by the implemented nearest-safe policy; they do not demonstrate that residents would select the same alternatives in practice.

7.4 Flood-Aware Commuting and Levy Walk V2

Levy Walk V2 replaces opportunistic legacy movement with explicit worker and student attendance. Cycle-level targets are allocated exactly across homes and hourly departure profiles. Trips are divided into packets of no more than 50 people. The plugin audits demand suppressed at disabled origins, unavailable or imminently flooded destinations, temporary return homes, blocked returns, and later repatriation.

Stage 6 contains five 13-region families with seeds 0–29 and 56 simulated days. All 150 runs pass both the Popular Times and Levy Walk V2 validations. Table 12 shows that commute fulfillment falls under each flood mode and is lowest when all node types are exposed. Popular Times rerouting restores activity visits in the combined condition but does not restore work and school attendance.

Cross-version results are labeled combined model revisions. Levy Walk V2 changes commute demand and lifecycle semantics, and destination flooding now includes work and school. Similarity in the shared Popular Times metrics therefore checks compatibility, but differences do not isolate an algorithmic improvement over the legacy Levy action.

Table 12 Stage 6 outcomes over 30 matched seeds; brackets contain 95% intervals.

Scenario	Commute ment	fulfill-	Mean commute dis- tance (m)	Activity fulfillment
No flood	1.0000 [1.0000–1.0000]		1,393.53 [1,392.87– 1,394.19]	1.0000
Destinations flooded	0.9354 [0.9352–0.9355]		1,343.80 [1,343.03– 1,344.56]	0.9649
Homes flooded	0.9406 [0.9404–0.9408]		1,390.89 [1,390.10– 1,391.69]	1.0000
All node types flooded	0.8915 [0.8912–0.8917]		1,335.82 [1,335.02– 1,336.62]	0.9649
All flooded, POIs rerouted	0.8905 [0.8903–0.8908]		1,335.82 [1,334.98– 1,336.67]	1.0000

7.5 Mobility Synthesis

The three stages separate disruption from response. Stage 4 quantifies unmet demand when unavailable destinations are simply suppressed. Stage 5 adds an explicit substitution rule and measures its travel and receiving-load consequences. Stage 6 shows that the same rule can restore activity visits while assigned work and school trips remain disrupted. The broader result is not that one category is intrinsically discretionary, but that substitutability must be represented explicitly for each destination type. A model that silently replaces all unavailable destinations would hide this assumption and overstate adaptation.

8 Visualization and Exploratory Analysis

8.1 Simulation Output and Origin–Destination Data

LodusPop records complementary views of a run. Population and node-state logs describe occupancy, characteristics, effective availability, and disable causes over time. Movement records retain origin, destination, step, and moved population. Domain loggers add demand, admission, queue, rerouting, and unmet-reason events. These records can be aggregated into node, region, census-sector, and origin–destination tables without changing the simulated state.

An origin–destination matrix reports how much population moved between two units during a step, cycle, or complete run. It does not encode the route between their representative coordinates. This distinction is important for interpreting both the immersive and geographic-flow tools.

8.2 Immersive Virtual-Reality Visualization

Teixeira et al. [22] developed a Unity application for the Meta Quest 2 that loads an OpenStreetMap-derived environment, monitored nodes, and formatted LODUS output. Users can inspect occupancy at the current step or over time, group regions of interest, and view moving healthy and infected populations within a three-dimensional map. Capacity-oriented displays highlight potential overcrowding.

The demonstration used 13 regions of interest, seven steps, and randomly generated displacement and infection events. It therefore evaluates whether the prototype can ingest and present the data format. It does not validate a hospital-demand process, contamination model, or planning intervention. The study itself identifies controlled user studies as future work for usability and analytical effectiveness.

8.3 Interactive Geographic Flow Visualization

The student-led geographic flow tool [23] uses React, TypeScript, MapLibre, and DuckDB-WASM to query and display origin–destination data in the browser. It supports selection of territorial units, incoming and outgoing flows, density controls, demographic filters, coordinated indicators and charts, multiscale aggregation, reusable configuration profiles, and local caching.

Its principal evaluation uses 2021 England and Wales census commuting at Middle Layer Super Output Area and Lower Tier Local Authority levels. A simulated Porto Alegre dataset tests reuse with another geography. Lines connect territorial centroids and therefore show aggregate relationships rather than traveled paths. The reported latency evaluation is exploratory and does not replace a controlled performance or user study.

8.4 Applicability to the Recent Experiments

The recent loggers already produce compatible aggregate relations: home–clinic flows for dialysis, origin–hospital flows for inpatient care, home–POI flows for Popular Times, and home–work or home–school flows for Levy Walk V2. Node-state records can additionally identify when a destination was disabled and why. These interfaces make integration with the geographic-flow tool feasible and allow immersive views to animate formatted transfers.

No completed study in this thesis evaluates those recent batches inside either visualization application. Their compatibility is therefore presented as a supported integration path, not as an experimental result. A future evaluation should define analytical tasks, compare views, recruit intended users, and measure accuracy, time, confidence, and usability.

8.5 Capabilities, Limitations, and Author Contributions

Visualization supports inspection, communication, and anomaly discovery. It can reveal concentrated flows, unexpected receivers, overcrowded nodes, and changes over time. It cannot establish that the generating assumptions match observed behavior. Geographic aggregation can conceal within-area variation, centroid lines can imply nonexistent routes, and three-dimensional animation can give synthetic data an unwarranted appearance of realism.

The immersive prototype is collaborative published work by Teixeira, the thesis author, and their coauthors; this thesis does not claim that it was developed solely by the thesis author. The geographic flow application and manuscript were developed by Gustavo Willian Martins da Silva as student-led work under the thesis author’s

supervision. The present thesis contributes the integration and critical synthesis of these artifacts with the recent LodusPop outputs.

9 Discussion

9.1 Answers to the Research Questions

RQ1—representation across granularities. The same region, point-of-interest, blob, template, and action abstractions support both the original neighborhood scenario and the recent census-sector home distributions. Moving from 94 aggregated homes to 435 or 2,711 census-sector homes changes the input graph and possible travel distances, not the population model. The experiments therefore demonstrate representational reuse. They do not establish that every spatial resolution is equally accurate; the appropriate resolution remains constrained by data and the phenomenon being studied.

RQ2—extension without changing the core. Disease states, hospital admissions, recurring dialysis demand, activity visits, commuting, and flood response are implemented by plugins consuming the same population and environment interfaces. Cause-aware availability and dependencies provide the common environmental state needed by several plugins. Capacity, queues, rerouting, and return lifecycles remain domain-specific. This boundary is the principal architectural answer to RQ2.

RQ3—versatility and limits. The original and recent studies cover substantially different timescales, destination types, population selections, and outcome metrics. Their successful execution and invariant checks show that the framework can express these scenarios. The health and flood results also produce interpretable within-model contrasts. Versatility is therefore supported as software and modeling capability; predictive validity is not established across the application domains.

RQ4—auditable interpretation. Explicit provenance, matched seeds, deterministic/stochastic separation, demand accounting, node-state logs, conservation checks, and domain event logs make scenario outcomes traceable. Visualization can help inspect the records, but it is not a substitute for calibration or external validation. The strongest interpretation is thus a reproducible comparison between named configurations.

9.2 Evidence of Framework Extensibility

Table 13 summarizes how the studies reuse the core while changing plugin-level mechanisms. No single metric spans all rows because each experiment represents a different construct. The relevant framework evidence is that populations remain conserved and traceable while the environment and plugins express each domain’s demand and response.

The plugin architecture also prevents successful behavior in one domain from silently becoming a general rule. The nearest-safe policy restores Popular Times visits, but it is not applied to dialysis, hospital admissions, workplaces, or schools unless a corresponding plugin explicitly defines that policy. This makes model composition more verbose, but it preserves the assumptions needed to audit a result.

Table 13 Cross-study synthesis of framework use, evidence, and limits.

Study	Spatial input	Extension or disruption	Response and metrics	Principal limit
Original cases	94 neighborhood homes	Levy movement, events, contagion, vaccination	Movement, infection, blob count, runtime, memory	Aggregated homes and no dynamic facility loss
Inpatient care	94 regions; census-sector homes	Bed pools, stays, disabled facilities, demand stress	Overflow or FIFO queue; coverage, delay, distance	Synthetic demand elements and horizon censoring
Dialysis	Reduced and complete census-sector environments	Recurring demand, outages, recovery, water dependencies	Admission to enabled capacity; coverage, overdue demand, distance	Configured demand and disruption schedules
Popular Times	13-region flood matrix; 94-region no-flood baseline	Weekly flooded homes and POIs	Suppression or nearest-safe rerouting; fulfillment and displacement	Fixed profiles and unvalidated substitution behavior
Levy Walk V2	13 regions; census-sector homes	Work/school attendance and flood-aware returns	Commute fulfillment, unmet reasons, distance, repatriation	Assigned destinations and combined cross-version changes
Visualization	Node, temporal, and OD outputs	Immersive and multiscale geographic views	Exploration of occupancy and flows	Synthetic/exploratory evaluations, not validation

9.3 Spatial Resolution and Data Availability

The meaning of a region and point of interest is selected by the simulation designer. A point may represent a specific building, all facilities of a type in a neighborhood, or an abstract location at a broader scale. Distances and domain parameters must be interpreted consistently with that choice. A contagion parameter calibrated for building-scale contact, for example, cannot be transferred unchanged to movement between cities or states.

The recent census-sector environments make the flexibility discussed in the original manuscript concrete. Multiple home nodes now occur within one neighborhood, allowing sources and flood exposure to vary within the region. This improves spatial expression but increases input size and does not automatically improve behavioral validity. Census and municipal datasets may be outdated or incomplete, and sampled characteristics may omit joint dependencies between age, occupation, payer, and mobility. Resolution should therefore be justified by the research question and source data rather than maximized as an end in itself.

9.4 Capacity, Failure Topology, Dependencies, and Adaptation

The health experiments show that availability and capacity answer different questions. The inpatient overflow policy reveals latent shortage, while the queue policy represents the consequence of enforcing capacity. Dialysis failures show that nominal capacity loss can be temporary, persistent, concentrated, distributed, or caused indirectly through a dependency. Coverage alone is insufficient: two conditions with complete treatment coverage can impose different travel burdens.

The mobility experiments likewise separate exposure and response. Disabling a destination suppresses demand under one policy and invokes substitution under another. Restoring activity-visit fulfillment adds displacement and receiving load and does not repair assigned work or school trips. These results favor reporting a vector of outcomes—coverage or fulfillment, delay or unmet demand, travel, and load—over a single composite resilience score whose weights would introduce additional assumptions.

9.5 Visualization as Interpretation Rather Than Validation

Immersive and geographic visualizations expose temporal occupancy and spatial flows that are difficult to interpret in raw tables. They are especially relevant for investigating redistributed clinic demand, hospital catchments, replacement POIs, and commute disruptions. However, a coherent animation or map shows that records can be rendered, not that the generating process is realistic. Validation requires an independent observed quantity and a defined comparison criterion. The synthetic VR demonstration and exploratory OD-tool evaluation are therefore presented as supporting capabilities.

9.6 Threats to Validity and Reproducibility

Construct validity. Coverage, fulfillment, overflow, queue length, and distance approximate different dimensions of service or mobility. They do not measure welfare, clinical outcome, equity, or individual preference. Fixed visit profiles and assigned destinations simplify adaptive behavior.

Internal validity. Automated invariants and matched scenarios reduce implementation and comparison errors, but they cannot prove the absence of defects. Several outcomes are deterministic consequences of fixed demand and node schedules. Confidence intervals over seeds quantify only the stochastic components of the implementation.

External validity. Facility and geographic data anchor the scenarios in Porto Alegre, but many demand values, thresholds, failures, mappings, and alternatives are configured or synthetic. The results should not be extrapolated as forecasts. In particular, Santa Ana capacity, CID demand, and CID mappings are demonstrations, and the nearest-safe POI rule has not been compared with observed post-disruption choices.

Statistical validity. The batches use different summaries appropriate to their preserved reports: inpatient results provide P05–P95 ranges, dialysis analysis uses bootstrap intervals, and the repeated mobility studies use reported 95% t intervals.

Deterministic conditions are not assigned artificial intervals. Cross-domain metrics are not pooled, and the Stage 6 legacy comparison is not interpreted as an isolated algorithm effect.

Reproducibility. Recent configurations, raw outputs, run metadata, summaries, and validation artifacts are retained in the repository. The older LODUS-Health per-step distributions and seed were not retained, so its exact published timing is not reproducible; the new inpatient batch is the reproducible evidence used here. Software and data versions should be archived with the final thesis.

9.7 Future Work

The highest-priority methodological work is calibration against observed mobility and health-service data, sensitivity analysis for demand and flood thresholds, joint rather than independent population-characteristic distributions, and explicit uncertainty in external inputs. Health plugins could add transfers, dynamic load balancing, priority rules, and alternative service networks. Mobility plugins could estimate destination-choice policies from observed disruption data and represent transport-network accessibility rather than straight-line distance.

Framework work includes formal complexity analysis, controlled comparisons with MATSim, SUMO, GAMA, and other suitable baselines, richer composition rules for simultaneous plugins, and explicit agent-based micro-dynamics where localized interaction is required. Visualization work should include task-based user studies and a tested pipeline from recent simulation outputs to both tools.

A shelter-allocation workflow exists for the 94-region environment, including exposure, capacity, evacuation, reallocation, and shelter-failure configurations. No completed shelter batch summary is preserved in the repository, so shelter outcomes are not reported. They should enter the thesis only after the experiment catalog, validations, provenance, and results are completed to the same standard as the included studies.

10 Conclusion

10.1 Summary of Contributions

This thesis presented LodusPop, an environment-driven framework for modeling heterogeneous urban populations as blobs that can split, merge, move, and change traceable state. Regions, points of interest, population templates, routines, and actions provide the common model. Plugins extend action semantics without adding every domain concept to the core. The environment was extended with effective node availability, independent disable causes, and all-of or minimum-of dependency constraints.

The original published experiments were retained as the foundation of the evaluation. Recent studies then exercised the framework with census-sector homes, hospital and dialysis capacities, facility and infrastructure failures, flood exposure, activity demand, destination substitution, and flood-aware work and school attendance. The

thesis also situated immersive and origin–destination visualizations as supporting means of inspecting output.

10.2 Main Findings Across Application Domains

Across health services, modeled access depends on more than aggregate capacity. Enforcing capacity creates queues and delays hidden by unconstrained overflow; recovery time, spatial allocation, failure topology, and supporting infrastructure change coverage and travel. Across flood-aware mobility, origin and destination loss have different consequences, and adaptation depends on an explicit substitution policy. Nearest-safe rerouting restores the configured activity visits but does not compensate for unavailable assigned workplaces and schools.

These findings answer the thesis objective at the level supported by the evidence. LodusPop can express and audit a broad family of urban population scenarios through a stable population model and domain plugins. The studies demonstrate framework versatility and generate meaningful within-model contrasts. They do not establish that the scenarios forecast clinical demand, individual destination choice, or future disaster impacts.

10.3 Limitations and Research Outlook

The principal limitations concern calibration, input provenance, simplified demand and destination policies, and the absence of direct validation against observed mobility or service outcomes. Spatial detail is bounded by the available data, and finer input resolution does not remove behavioral uncertainty. Runtime measurements are implementation-, hardware-, and scenario-dependent and are not head-to-head benchmarks against other frameworks.

Future work should combine preserved, versioned experiment catalogs with observed data, sensitivity and uncertainty analysis, comparative simulators, richer health-service and transport-network policies, and controlled visualization studies. Subject to those limits, LodusPop provides a reusable foundation for explicit, auditable comparisons of urban population dynamics under routine conditions, capacity constraints, and disruption.

Acknowledgements. The authors acknowledge the use of ChatGPT, version GPT-4o, to review and enhance the text quality of the original manuscript that forms the basis of this thesis.

Declarations

Funding. This study was partly funded by: the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001; the Conselho Nacional de Desenvolvimento Científico e Tecnológico - Brasil (CNPq) - Process Number 309228/2021-2; the Fundação de Amparo à Pesquisa do Estado do Rio Grande do Sul (FAPERGS) - Process Number 20/2551-0000280-2; and the Instituto Nacional de Ciência e Tecnologia em Simulação e Monitoramento para Assistência ao Indivíduo em Eventos Climáticos Extremos (INCT-SiM-AI) - Process Number 408330/2024-4.

Competing Interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this thesis

Data availability. Data will be made available on request.

Code availability. Code will be made available on request.

Author contribution. **Gabriel Fonseca Silva:** Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing, Visualization. **André da Silva Antonitsch:** Conceptualization, Methodology, Investigation. **Soraia Raupp Musse:** Conceptualization, Methodology, Validation, Investigation, Writing – review & editing, Supervision, Project administration, Funding acquisition.

Appendix A Input and Simulation Setup

This Appendix provides examples of the data input and simulation setup required to execute simulations, as defined by our proposed model in Section 3. The setup includes:

- *i*) input files describing the environment, population, and routines (detailed in Appendices A.1, A.2 and A.3, respectively);
- *ii*) plugin scripts to introduce new action types into the simulation (Appendix A.4); and
- *iii*) a simulator script responsible for executing the simulation.

In the current implementation, both the simulator and plugin scripts are written in Python, and the input data is structured in the JSON file format, ensuring ease of use and extensibility.

A.1 Environment Input Files

The environment input file provides a detailed description of all regions (R_s) and their respective points of interest (P_s) in the simulation, in accordance with the definitions outlined in Section 3.1. Listing 1 illustrates an example environment configuration containing a single region named “*ExampleRegion*”, which includes three points of interest: a school, a pharmacy, and a home. Data to describe the environment can be obtained from Geographic Information System (GIS) databases such as OpenStreetMap, as presented in Figure 1.

Each element in the environment input file, whether a region or a point of interest, is characterized by an identifier and a geographical location. These attributes are specified using the *name* and *lng_lat* properties, respectively.

```
"regions": [  
  {  
    "name": "ExampleRegion",  
    "lng_lat": [-51.19757, -30.15647],  
    "points_of_interest": [  
      {  
        "name": "school",  
        "lng_lat": [-51.19750, -30.15621]      }  
    ]  
  }  
]
```

```

    },
    {
      "name": "pharmacy",
      "lng_lat": [-51.19820, -30.15655]
    },
    {
      "name": "home",
      "lng_lat": [-51.19805, -30.15633]
    }
  ]
}
]

```

Listing 1 Example of an environment input file.

A.2 Population Input Files

The population input file provides a detailed description of all blobs (B s), including their characteristics and locations at the start of the simulation, in alignment with the definitions in Section 3.2. Listing 2 presents an example population input containing two initial blobs. Population data can be sourced from census data (Real Population) or generated synthetically (Synthetic Population) to meet the specific requirements of an experiment, as shown in Figure 1.

The *default_traceable_characteristics* property sets default values for the Tc s of all blobs. Unless explicitly defined in their setup, all blobs start the simulation with a *VaccinationStatus* traceable characteristic set to *Unvaccinated*.

The *sampled_characteristics_categories* property specifies the possible categories for each sampled characteristic used by all blobs in the simulation. However, the population distribution across these categories is defined individually for each blob, as described below.

The *initial_population* property outlines points of interest and their initial populations (i.e., B_P). Points of interest are identified by their containing regions and identifiers, such as *ExampleRegion//home*, which references the *home* P within *ExampleRegion*, as described in Listing 1. A point of interest may include multiple blob descriptions, but only those with an initial population need to be included in the input file.

Each P within *initial_population* contains a vector of elements describing blobs, with each element providing:

- The total population of a blob (TP);
- Optional values for Tc , which override the *default_traceable_characteristics*; and
- Values for all categories of the Sc .

For instance, in Listing 2, two blobs are created in the *home* P of *ExampleRegion*. Blob B_i has $TP_i = 100$, with its Tc_i equal to the default values, as no specific values were assigned in its *traceable_characteristics* property. Blob B_j , on the other hand, has $TP_j = 20$ and an initial *VaccinationStatus* traceable characteristic value of *Vaccinated*.

```

"default_traceable_characteristics": {
  "VaccinationStatus": "Unvaccinated",
  "Traceable_B": "Value_B"
},
"sampled_characteristics_categories": {

```

```

    "Age": [ "Children", "Adults", "Elders" ],
    "Sampled_B": [ "Value_B1", "Value_B2" ]
  },
  "initial_population": {
    "ExampleRegion//home": [
      {
        "total_population": 100,
        "traceable_characteristics": {},
        "sampled_characteristics": {
          "Age": { "Children": 20, "Adults": 50, "Elders": 30 },
          "Sampled_B": { "Value_B1": 43, "Value_B2": 57 }
        }
      },
      {
        "total_population": 20,
        "traceable_characteristics": {
          "VaccinationStatus": "Vaccinated"
        },
        "sampled_characteristics": {
          "Age": { "Children": 20, "Adults": 0, "Elders": 0 },
          "Sampled_B": { "Value_B1": 12, "Value_B2": 8 }
        }
      }
    ]
  }
}

```

Listing 2 Example of a population input file.

A.3 Routine Input Files

The routine input file provides a description of the Global Routine (*GRO*) and the local routines for individual points of interest, following the definitions outlined in Section 3.3. Listing 3 presents an example input containing routines for two points of interest: a *home* P_i and a *pharmacy* P_j , both located in the *ExampleRegion*.

P_i includes a single action, A_i , executed at $t = 8$ during each simulation cycle. A_i is of type *move population* (defined in Section 3.3) and moves 20 individuals—specifically unvaccinated children and young students—from P_i to the *pharmacy* P_j in *ExampleRegion*.

P_j , on the other hand, has two actions, A_j and A_k , performed at $t = 9$ and $t = 10$, respectively, during each simulation cycle. A_j is of type *change traceable characteristic* (detailed in Section 3.3), vaccinating 5 people currently at P_j . A_k , of type *move population*, transfers the vaccinated individuals from P_j back to P_i .

The *population_template* property specifies the profile of individuals selected for each action, such as students who are children or young adults. The *name* property, included as a label, does not influence the simulation directly but serves as a reference for annotating the purpose of the action. While Listing 3 does not include actions for the Global Routine, these follow the same structure as individual routines, encompassing a cycle step, action type, population template, and any additional parameters.

```

"global_routine": [],
"routines": {
  "ExampleRegion//home": [
    {
      "cycle_step": 8,
      "action": {
        "name": "send_students_to_pharmacy",
        "type": "move_population",
        "population_template": {
          "traceable_characteristics": {

```

```

        "VaccinationStatus": "Unvaccinated"
    },
    "sampled_characteristics": {
        "Age": [ "Children", "Youngs" ],
        "Occupation": [ "Student" ]
    }
},
"values": {
    "origin_region": "ExampleRegion",
    "origin_poi": "home",
    "destination_region": "ExampleRegion",
    "destination_poi": "pharmacy",
    "quantity": 20
}
}
},
"ExampleRegion//pharmacy": [
    {
        "cycle_step": 9,
        "action": {
            "name": "vaccinate_population",
            "type": "change_traceable_characteristic",
            "population_template": {
                "traceable_characteristics": {
                    "VaccinationStatus": "Unvaccinated"
                },
                "sampled_characteristics": {}
            },
            "values": {
                "characteristic_to_change": "VaccinationStatus",
                "new_value": "Vaccinated",
                "quantity": 5
            }
        },
    },
    {
        "cycle_step": 10,
        "action": {
            "name": "send_vaccinated_population_back",
            "type": "move_population",
            "population_template": {
                "traceable_characteristics": {
                    "VaccinationStatus": "Vaccinated"
                },
                "sampled_characteristics": {}
            },
            "values": {
                "origin_region": "ExampleRegion",
                "origin_poi": "pharmacy",
                "destination_region": "ExampleRegion",
                "destination_poi": "home",
                "quantity": 5
            }
        },
    }
]
}

```

Listing 3 Example of a routine input file.

A.4 Plugins

Plugins define additional action types and their corresponding functions while also providing external parameters and data. These resources enable simulation designers to tailor simulations and model diverse scenarios. Plugins are implemented as Python modules and imported by the simulator script, described in the following section.

Listing 4 shows an example plugin that processes the *"example_action"* action type. This action transfers 100 individuals from a point of interest P_i , specified by the *values* parameter, to a *home* P_j within the same region. The plugin accomplishes this by creating and returning a *move population* action, which is then executed by its corresponding function.

```

class ExamplePlugin(environment.TimeActionPlugin):

    def __init__(self, environment_graph):
        super().__init__()

```

```

self.graph = environment_graph
self.set_pair('example_action', self.consume_action)
self.example_external_parameter = 'foo'

def consume_action(self, pop_template, values, cycle_step, sim_step):
    region = self.graph.get_region(values['region'])
    poi = region.get_poi(values['node'])

    new_action_type = 'move_population'
    new_action_values = {}
    new_action_values['origin_region'] = region.name
    new_action_values['origin_poi'] = poi.name
    new_action_values['destination_region'] = region.name
    new_action_values['destination_poi'] = 'home'
    new_action_values['quantity'] = 100
    action = environment.TimeAction(_type = new_action_type,
                                    _pop_template = pop_template,
                                    _values = new_action_values)

    return [action]

```

Listing 4 Example of a plugin script that includes a "example_action" in the simulation.

A.5 Simulator script

The simulator script manages the parsing of input data, the integration of plugins, and the overall control of the simulation flow. Listing 5 presents an example of a simulator script. Parsing is handled by a function that takes input file paths as arguments and returns an *EnvGraph* framework class instance. Plugins are incorporated by initializing their classes and loading them into the *EnvGraph*. The simulation flow is governed by a loop that updates the simulation step at each iteration.

```

# Data Loading
env_graph = Generate_EnvironmentGraph([data_input_file_paths])

# Simulation Parameters
cycles = 3
cycle_duration = 24
env_graph.routine_cycle_length = cycle_duration
simulation_steps = cycles * cycle_duration

# Loading Plugins
ex_plugging = ExamplePlugin(env_graph)
ex_plugging.example_external_parameter = 'bar'
env_graph.LoadPlugin(ex_plugging)

# SIMULATION
for i in range(simulation_steps):
    # Updates POI Routines and Global Routines
    # These are defined by the input files
    env_graph.update_time_step(i % cycle_duration, I)

    # Actions can be directly called or queued in the simulation loop
    # if i == 60:
    #     dummy_action = TimeAction(_type = new_action_type,
    #                               _pop_template = dummy_template,
    #                               _values = dummy_values)
    #     env_graph.direct_action_invoke(dummy_action)
    #     env_graph.queue_next_frame_action(dummy_action)

```

Listing 5 Simulation script example.

References

- [1] Sharifi, A., Khavarian-Garmsir, A.R.: The covid-19 pandemic: Impacts on cities and major lessons for urban planning, design, and management. *Science of The Total Environment* **749**, 142391 (2020) <https://doi.org/10.1016/j.scitotenv.2020.142391>
- [2] Kang, B.G., Park, H.-M., Jang, M., Seo, K.-M.: Hybrid Model-Based simulation analysis on the effects of social distancing policy of the COVID-19 epidemic. *Int J Environ Res Public Health* **18**(21) (2021)
- [3] Bouchnita, A., Chekroun, A., Jebrane, A.: Mathematical modeling predicts that strict social distancing measures would be needed to shorten the duration of waves of COVID-19 infections in vietnam. *Front Public Health* **8**, 559693 (2021)
- [4] Eid, M.S., Eldin, H.K.: Simulation models as a tool for urban planning. *Computers & Industrial Engineering* **4**(1), 53–66 (1980) [https://doi.org/10.1016/0360-8352\(80\)90039-X](https://doi.org/10.1016/0360-8352(80)90039-X)
- [5] Koumetio Tekouabou, S.C., Diop, E.B., Azmi, R., Jaligot, R., Chenal, J.: Reviewing the application of machine learning methods to model urban form indicators in planning decision support systems: Potential, issues and challenges. *Journal of King Saud University - Computer and Information Sciences* **34**(8, Part B), 5943–5967 (2022) <https://doi.org/10.1016/j.jksuci.2021.08.007>
- [6] Haraguchi, M., Nishino, A., Kodaka, A., Allaire, M., Lall, U., Kuei-Hsien, L., Onda, K., Tsubouchi, K., Kohtake, N.: Human mobility data and analysis for urban resilience: A systematic review. *Environment and Planning B: Urban Analytics and City Science* **49**(5), 1507–1535 (2022) <https://doi.org/10.1177/23998083221075634>
- [7] Serena, L., Marzolla, M., D’Angelo, G., Ferretti, S.: A review of multilevel modeling and simulation for human mobility and behavior. *Simulation Modelling Practice and Theory* **127**, 102780 (2023) <https://doi.org/10.1016/j.simpat.2023.102780>
- [8] Helbing, D., Molnar, P.: Social force model for pedestrian dynamics. *Physical Review E* **51**(5), 4282 (1995)
- [9] Cao, R.F., Lee, E.W.M., Yuen, A.C.Y., Chen, T.B.Y., De Cachinho Cordeiro, I.M., Shi, M., Wei, X., Yeoh, G.H.: Simulation of competitive and cooperative egress movements on the crowd emergency evacuation. *Simulation Modelling Practice and Theory* **109**, 102309 (2021) <https://doi.org/10.1016/j.simpat.2021.102309>
- [10] Khan, S., Deng, Z.: Agent-based crowd simulation: an in-depth survey of determining factors for heterogeneous behavior. *The Visual Computer* **40**(7),

- [11] Ruprecht, I., Michelic, F., Eggeling, E., Preiner, R.: Adaptive movement behavior for real-time crowd simulation. *The Visual Computer* **40**(7), 4789–4803 (2024) <https://doi.org/10.1007/s00371-024-03476-2>
- [12] Hughes, R.L.: A continuum theory for the flow of pedestrians. *Transportation Research Part B: Methodological* **36**(6), 507–535 (2002)
- [13] Treuille, A., Cooper, S., Popović, Z.: Continuum crowds. *ACM Transactions on Graphics* **25**(3), 1160–1168 (2006)
- [14] Thalmann, D., Musse, S.R.: *Crowd Simulation*. Springer, London (2007). <https://doi.org/10.1007/978-1-84628-825-8>
- [15] Xiong, M., Lees, M., Cai, W., Zhou, S., Low, M.Y.H.: Hybrid modelling of crowd simulation. *Procedia Computer Science* **1**(1), 57–65 (2010)
- [16] Ijaz, K., Sohail, S., Hashish, S.: A survey of latest approaches for crowd simulation and modeling using hybrid techniques. In: *Proceedings of the 2015 17th UKSIM-AMSS International Conference on Modelling and Simulation. UKSIM '15*, pp. 111–116. IEEE Computer Society, USA (2015)
- [17] Antonitsch, A.D.S., Schaffer, D.H.M., Rockenbach, G.W., Knob, P., Musse, S.R.: BioClouds: A multi-level model to simulate and visualize large crowds. In: *Computer Graphics International Conference*, pp. 15–27 (2019). Springer
- [18] Dang, H.-T., Gaudou, B., Verstaëvel, N.: A multi-level density-based crowd simulation architecture. In: Mathieu, P., Dignum, F., Novais, P., Prieta, F. (eds.) *Advances in Practical Applications of Agents, Multi-Agent Systems, and Cognitive Mimetics. The PAAMS Collection*, pp. 64–75. Springer, Cham (2023)
- [19] Cogo, E., Prazina, I., Hodzic, K., Haseljc, H., Rizvic, S.: Survey of integrability of procedural modeling techniques for generating a complete city. In: *2019 XXVII International Conference on Information, Communication and Automation Technologies (ICAT)*, pp. 1–6 (2019). <https://doi.org/10.1109/ICAT47117.2019.8939012>
- [20] Alessandretti, L., Aslak, U., Lehmann, S.: The scales of human mobility. *Nature* **587**(7834), 402–407 (2020) <https://doi.org/10.1038/s41586-020-2909-1>
- [21] Souza, G.V.d.: *LODUS-Health: A tool for simulating the flow of patients in hospital beds in porto alegre*. Master’s dissertation, Pontifical Catholic University of Rio Grande do Sul, Porto Alegre, Brazil (2026)
- [22] Teixeira, V.C., Silva, G.F., Manssour, I.H., Musse, S.H., Pinho, M.S.: Visualization of crowd contamination simulations using immersive virtual reality.

- In: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), pp. 1819–1824. IEEE, ??? (2024). <https://doi.org/10.1109/COMPSAC61105.2024.00287>
- [23] Silva, G.W.M.d., Silva, G.F.: Visualization of Geospatial Mobility Data. Student research manuscript, Pontifical Catholic University of Rio Grande do Sul (2026)
 - [24] Van Den Berg, J., Guy, S.J., Lin, M., Manocha, D.: Reciprocal n-Body Collision Avoidance. In: Robotics Research, pp. 3–19. Springer, ??? (2011)
 - [25] Lima Bicho, A., Rodrigues, R.A., Musse, S.R., Jung, C.R., Paravisi, M., Magalhães, L.P.: Simulating crowds based on a space colonization algorithm. *Computers & Graphics* **36**(2), 70–79 (2012)
 - [26] Pelechano Gómez, N., O’Brien, K., Silverman, B.G., Badler, N.: Crowd simulation incorporating agent psychological models, roles and communication. In: First International Workshop on Crowd Simulation, pp. 21–31 (2005)
 - [27] Durupinar, F., Gdkbay, U., Aman, A., Badler, N.I.: Psychological parameters for crowd simulation: From audiences to mobs. *IEEE Transactions on Visualization and Computer Graphics* **22**(9), 2145–2159 (2015)
 - [28] Neto, A.B.F., Pelachaud, C., Musse, S.R.: Giving emotional contagion ability to virtual agents in crowds. In: International Conference on Intelligent Virtual Agents, pp. 63–72 (2017). Springer
 - [29] Antonitsch, A., Musse, S.R., Figueiredo, L.H.: Towards a legion of virtual humans: Steering behaviors and organic visualization. In: SIBGRAPI Conference on Graphics, Patterns and Images, pp. 31–38 (2020). IEEE
 - [30] Lemonari, M., Panayiotou, A., Kyriakou, T., Pelechano, N., Chrysanthou, Y., Aristidou, A., Charalambous, P.: Mpack: Mesoscopic profiling and abstraction of crowd trajectories. *Computer Graphics Forum* **n/a**(n/a), 70156 <https://doi.org/10.1111/cgf.70156> <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.70156>
 - [31] Alvarez, H.V., Patsatzis, D.G., Russo, L., Kevrekidis, I., Siettos, C.: Next Generation Equation-Free Multiscale Modelling of Crowd Dynamics via Machine Learning (2025). <https://arxiv.org/abs/2508.03926>
 - [32] Li, Y., Chen, Y., Liu, J., Huang, T.: Efficient crowd simulation in complex environment using deep reinforcement learning. *Scientific Reports* **15**(1), 5403 (2025) <https://doi.org/10.1038/s41598-025-88897-2>
 - [33] Brockmann, D., Hufnagel, L., Geisel, T.: The scaling laws of human travel. *Nature* **439**(7075), 462–465 (2006)
 - [34] Gonzalez, M.C., Hidalgo, C.A., Barabasi, A.-L.: Understanding individual human

- mobility patterns. *Nature* **453**(7196), 779–782 (2008)
- [35] Song, C., Koren, T., Wang, P., Barabási, A.-L.: Modelling the scaling properties of human mobility. *Nature Physics* **6**(10), 818–823 (2010) <https://doi.org/10.1038/nphys1760>
 - [36] Jamshidi, S., Baniasad, M., Niyogi, D.: Global to USA county scale analysis of weather, urban density, mobility, homestay, and mask use on COVID-19. *International Journal of Environmental Research and Public Health* **17**(21), 7847 (2020)
 - [37] Hernández-Orallo, E., Armero-Martínez, A.: How human mobility models can help to deal with COVID-19. *Electronics* **10**(1), 33 (2021)
 - [38] Kraemer, M.U.G., Yang, C.-H., Gutierrez, B., Wu, C.-H., Klein, B., Pigott, D.M., Plessis, L., Faria, N.R., Li, R., Hanage, W.P., Brownstein, J.S., Layan, M., Vespignani, A., Tian, H., Dye, C., Pybus, O.G., Scarpino, S.V.: The effect of human mobility and control measures on the covid-19 epidemic in china. *Science* **368**(6490), 493–497 (2020) <https://doi.org/10.1126/science.abb4218>
 - [39] Silva, G.F., Cassol, V., Neto, A.B.F., Antonitsch, A., Schaffer, D., Musse, S.R., Marsillac Linn, R.: LODUS: A multi-level framework for simulating environment and population - A contagion experiment on a pandemic world. In: *IEEE International Smart Cities Conference*, pp. 1–8 (2020). IEEE
 - [40] Liu, J., Xu, L., Ma, L., Chen, N.: Modeling population mobility flows: A hybrid approach integrating a gravity model and machine learning. *ISPRS International Journal of Geo-Information* **13**(11) (2024) <https://doi.org/10.3390/ijgi13110379>
 - [41] Guo, H., Zhang, W., Yang, J., Hou, Y., Dong, L., Liu, Y.: Data driven discovery of human mobility models (2025). <https://arxiv.org/abs/2501.05684>
 - [42] Du, Y., Feng, J., Yuan, J., Li, Y.: CAMS: A CityGPT-Powered Agentic Framework for Urban Human Mobility Simulation (2025). <https://arxiv.org/abs/2506.13599>
 - [43] Balmer, M., Rieser, M., Meister, K., Charypar, D., Lefebvre, N., Nagel, K.: Matsim-t: Architecture and simulation times. In: *Multi-agent Systems for Traffic and Transportation Engineering*, pp. 57–78. IGI Global Scientific Publishing, Hershey, PA (2009)
 - [44] Krajzewicz, D., Erdmann, J., Behrisch, M., Bieker-Walz, L.: Recent development and applications of sumo - simulation of urban mobility. *International Journal On Advances in Systems and Measurements* **3 and 4** (2012)
 - [45] Taillandier, P., Gaudou, B., Grignard, A., Huynh, Q.-N., Marilleau, N., Caillou, P., Philippon, D., Drogoul, A.: Building, composing and experimenting complex

- spatial models with the gama platform. *GeoInformatica* **23**(2), 299–322 (2019) <https://doi.org/10.1007/s10707-018-00339-6>
- [46] El-Bouri, R., Eyre, D.W., Watkinson, P., Zhu, T., Clifton, D.A.: Hospital admission location prediction via deep interpretable networks for the year-round improvement of emergency patient care. *IEEE Journal of Biomedical and Health Informatics* **25**(1), 289–300 (2021) <https://doi.org/10.1109/JBHI.2020.2990309>
 - [47] Bruballa, E., Wong, A., Rexachs, D., Luque, E.: An intelligent scheduling of non-critical patients admission for emergency department. *IEEE Access* **8**, 9209–9220 (2020) <https://doi.org/10.1109/ACCESS.2019.2963049>
 - [48] Karbouband, K., Tabaa, M.: Bed allocation optimization based on survival analysis, treatment trajectory and costs estimations. *IEEE Access* **11**, 31699–31715 (2023) <https://doi.org/10.1109/ACCESS.2023.3260184>
 - [49] Allihaibi, W.G., Masoud, M., Elhenawy, M., Liu, S.Q., Burke, J., Karim, A.: Solving the emergency care patient pathway by a new integrated simulation-optimisation approach. *IEEE Access* **9**, 100895–100910 (2021) <https://doi.org/10.1109/ACCESS.2021.3096263>
 - [50] Ye, X., Du, J., Han, Y., Newman, G., Retchless, D., Zou, L., Ham, Y., Cai, Z.: Developing human-centered urban digital twins for community infrastructure resilience: A research agenda. *Journal of Planning Literature* **38**(2), 187–199 (2023) <https://doi.org/10.1177/08854122221137861>
 - [51] Kermack, W.O., McKendrick, A.G.: A contribution to the mathematical theory of epidemics. *Proceedings of the royal society of london. Series A, Containing papers of a mathematical and physical character* **115**(772), 700–721 (1927)
 - [52] Dutra, C.: Estimate of the basic number of reproduction r_0 of covid-19 in south american countries. *InterAmerican Journal of Medicine and Health* **3**, 1–7 (2020) <https://doi.org/10.31005/iajmh.v3i0.127>
 - [53] Souza, W.M., Buss, L.F., Candido, D.d.S.: Epidemiological and clinical characteristics of the covid-19 epidemic in brazil. *Nature Human Behaviour* **4**(8), 856–865 (2020) <https://doi.org/10.1038/s41562-020-0928-4>
 - [54] Franco, C.M.R., Dutra, R.F.: Modelo sir para propagação da covid-19 no estado da paraíba (brasil). *INTERMATHS* **2**(2), 39–48 (2021) <https://doi.org/10.22481/intermaths.v2i2.9696>